Manuel du langage `music-score` (`.mus`)

**Manuel du
 langage `music-score`
(`.mus`)**

- Introduction
  - Note pour produire des images pour ce manuel
  - Le langage music-score (`mus`)
    - Production de l'image
    - Détail du code
    - Code par paragraphe
  - Rognage automatique de l'image
  - Images produites
  - Dossier et nom des images produites
- Inclusions
- Statistiques
  - Remarques sur les statistiques
  - Le Tempo
- Options principales
  - Résumé des options
- Détail des options
  - Tonalité de la pièce
  - Numérotation des pages
  - Portées multiples
    - Espacement entre les systèmes
    - Espacement entre les portées
    - Taille des portées
    - Numérotation des mesures
    - Nommage des portées
    - Groupement des portées
      - Groupes à l'intérieur d'un même système

        Cas spécial du PIANO
    Fusion (merge) des silences
        Arrêt ponctuel de la fusion des silences
    Fichier de sortie MIDI
    Espacement entre les notes (`proximity`)
Commandes
    Commande de déclenchement de la compilation (START/STOP)
        Nom du fichier de l'image (définition explicite)
Notation LilyPond simplifiée
    Chiffrage/harmonisation/Analyse mélodique
    Répétitions
        Marque de répétition avec le signe pourcentage
        Répétition d'une note avec `<note>*N`
        Répétition d'un code avec `% ... %N`
    Les Barres
        1re, 2e, etc. fois dans les reprises
    Clé de l'expression
    Tonalité de l'expression (armure et changement)
    Transposition
        Instruments transpositeurs
    Triolet, quintolet et septolet
    Ornements et signes d'interprétation
    Signes d'interprétation
    Trilles
        Petites notes (grace notes)
            Problèmes des petites notes dans les fichiers MIDI
        Indication de l'octave exacte après une petite note
    Regroupement des hampes
    Notes « mergées »
    Arpège vers accord tenu
    Changement de portée
    Note jouée par autre main (piano)
    Marques d'octave
    Arpèges
        Notes connectés par \arp
    Doigtés
        Parenthèses
    Fonctions ruby (mode expert)
        Constantes notes

Parenthèses dans les arguments
Langage LilyPond (aide mémoire)
Altérations
Accords
Liaisons
Attache des hampes des notes
Anacrouse
Voix simultanées
Silence sur mesure entière
Placement des silences
Variable (aka « Definitions »)
Déclaration de la variable-définition
Variable dynamique
Hauteur de la variable
Répétition de la variable
Utilisation d'un rang de variables
Définition par section
NOTA BENE
Annexe
Fichier de coloration syntaxique pour Sublime Text
Étapes pour « arpège vers accord tenu »
Score Builder
Tests
Collisions verticales

Introduction

Note pour produire des images pour ce manuel

Pour produire facilement des images pour ce manuel :

- ouvrir un Terminal au dossier des images du manuel,
- jouer la commande :

```
score-i[TAB] << 'MUS'
# ici le code mus
# dont :

-> <nom-a-donner-a-image>
<code mus>
MUS
```

- récupérer l'image dans le dossier `scores` du dossier images du manuel,
- la glisser à l'endroit voulu dans ce manuel.

> *Note : Les apostrophes autour de « MUS » permettent de ne pas avoir d'évaluation du texte. Sans eux, on serait obligé d'échapper les caractères spéciaux, par exemple :*
>
> ```
> << { c e } \\\\ { g g } >>
> ```
>
> *Grâce aux apostrophes, on peut écrire l'expression exacte :*
>
> ```
> << { c e } \\ { g g } >>
> ```

Le langage music-score (`mus`)

Le langage `music-score` (maintenant la commande **`score-image`**) est un langage de programmation qui permet de produire très facilement des images de partitions simples (simple portée ou portée piano — pour le moment) en utilisant dans son moteur le langage LiliPond.

Une page en `music-score` (langage **mus**) peut ressembler à :

```
# Dans ma-musique.mus
--barres
--time
--piano

mes12==
```

```
a'8 b cis d cis4 cis
<a, cis e>1
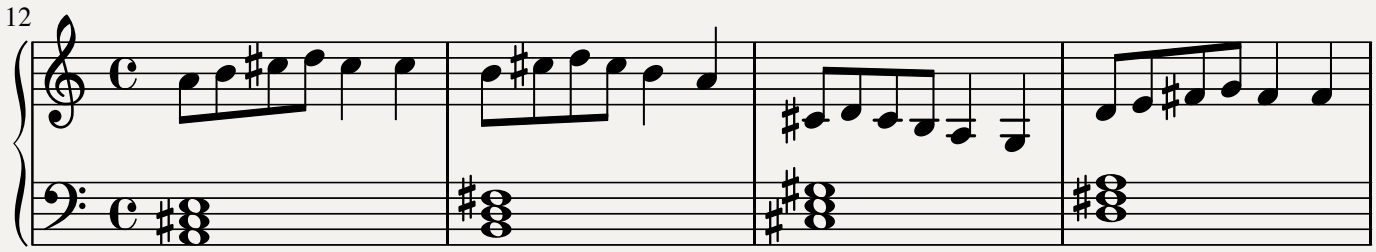
mes13==
b'8 cis d cis b4 a
<b, d fis>1

mes14==
cis8 d cis b a4 g
<cis, e gis>1

mes15==
d8 e fis g fis4 fis
<d, fis a>1

-> partition-12a15
--mesure 12
--proximity 7
mes12<->15
mes12<->15

```



Production de l'image

Pour produire l'image issue de ce code, on a utilisé la commande `score-image` en ligne de commande avec un *heredoc*. Comme ceci :

```
> score-image << MUS
--barres
--time
--piano
```

```
5
6  mes12==
7  a'8 b cis d cis4 cis
8  <a, cis e>1
9
10 mes13==
11 b'8 cis d cis b4 a
12 <b, d fis>1
13
14 mes14==
15 cis8 d cis b a4 g
16 <cis, e gis>1
17
18 mes15==
19 d8 e fis g fis4 fis
20 <d, fis a>1
21
22 -> partition-12a15
23 --mesure 12
24 --proximity 7
25 mes12<->15
26 mes12<->15
27 MUS
```

> *Noter que pour taper* ***`score-image`*** *il suffit de taper* `score-i` *puis la touche tabulation. C'est valable pour toutes les applications de la suite Score (Score-Builder, Score-Numbering, etc.).*

Détail du code

```
1
2  # Options préliminaires
3  # ---------------------
```

```
# Option pour ouvrir le fichier après fabrication
--open
# Option pour afficher les barres de mesure
--barres
# Option pour afficher la métrique
--time
# ou (pour ne pas le mettre)
--time OFF
# ou (pour la préciser)
--time 3/4
# Option indiquant qu'il s'agit d'une partition de piano
--piano

# Définition des mesures
# ----------------------
# Définition de la mesure 12
# La première ligne contient la main droite
# La seconde ligne définit la main gauche
mes12=
a'8 b cis d cis4 cis
<a, cis e>1

mes13=
etc.

# Définition des images (systèmes)
# --------------------------------
# Le nom de l'image SVG (affixe)
-> partition-12a15
# Le numéro de mesure à indiquer au début
--mesure 12
# L'éloignement horizontal entre les notes
--proximity 7
# Indique de la mesure 12 à la mesure 15, à la
# main gauche et à la main droite
mes12<->15
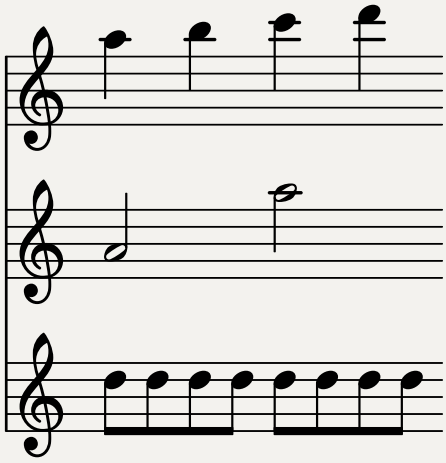
```

Code par paragraphe

Il est important de comprendre que ce code fonctionne **par paragraphe**, ce qui signifie que les éléments sont considérés comme « entier » lorsqu'ils ne sont séparés par aucune ligne vierge.

Cela permet entre autres choses d'avoir des partitions multipistes. Par exemple, un trio sera indiqué par :

```
# trio
-> trio
a b c d
a2.   a'
d8 d d d d d
```

… qui produira :



> *Tous les exemples de code donnés dans ce manuel font l'objet d'un test « checksum ». Pour lancer tous ces tests, il suffit de jouer en ligne de commande :* `score-image tests -dir=manuel`.

Cela comporte quelques inconvénients :

Parmi les inconvénients, il faut faire attention à bien séparer les définitions. Par exemple :

```
mesure1=
a b c d
mesure2=
e f g f
```

… ne produira pas les deux variables `mesure1` et `mesure2`, mais seulement la variable `mesure1` avec une erreur de `mesure2` inconnue.

Seule tolérance, pour la définition des noms d'image, reconnaissable à `->` en début de ligne :

```
mesure1=
a b c d
-> image_tolerable
mesure1
```

Le code ci-dessus sera considéré comme :

```
mesure1=
a b c d

-> image_tolerable
mesure1
```

Rognage automatique de l'image

Après la production du code, l'image est automatiquement rognée par **Inskape** pour ne laisser aucun air autour.

Images produites

Un unique fichier `.mus` peut définir une ou plusieurs images. Par exemple, un fichier contenant le code suivant :

```
# in durees.mus

-> image1
c d e f

-> image2
c2 d e f

-> image3
c1 d e f
```

… produira 3 images contenant leur code respectif, `durees/image1.svg`, `durees/image2.svg` et `durees/image3.svg`.

Dossier et nom des images produites

Par défaut (car il est possible de le déterminer explicitement), le nom des images et le dossier de leur destination sont définis par le nom du fichier `.mus` contenant le code music-score.

Soit le nom de fichier **`partition.mus`** contenant le code *music-score*.

Un dossier **`partition`** sera créé au même niveau que ce fichier, et contiendra les images produites. Chaque ligne contenant du code *mus* produira une image.

Les images porteront le nom **`partition-1.svg`**, **`partition-2.svg`**… **`partition-N.svg`** et seront placées dans le dossier `partition` ci-dessus.

Inclusions

Dans les fichiers `.mus`, on peut inclure d'autres fichiers `.mus` à l'aide de la commande :

```
INCLUDE path/to/musFile
```

Le chemin `path/to/musFile` peut être relatif au fichier maitre ou relatif au dossier `libmus` de l'application ***Score-Image*** qui définit des librairies standards.

La première librairie à avoir été créée est la librairie **`piano/Alberti.mus`** qui définit toutes les basses d'Alberti dans des variables.

Un fichier inclus peut définir n'importe quoi, pourvu que ce soit du code `.mus` à commencer par :

- des options,
- des variables,
- des partitions.

Pour la gestion des variables, voir Gestion dynamique des variables.

Statistiques

Le programme permet aussi de faire des **statistiques sur les notes** (nombre et durée). Il suffit :

- d'ajouter l'option `-s`/`--stats` à la ligne de commande (ou de mettre l'option `--stats` dans le fichier mus,
- de definir l'option `-t`/`--tempo=<val>[T]` en ligne de commande ou dans le code mus, pour calculer les durées réelles.

L'option et le tempo peut s'ajouter aussi directement dans le fichier **`.mus`** avec l'option :

```
--stats
--tempo 60T
```

Pour le tempo, voir ici.

Ces options produisent un dossier `stats` contentant 4 fichiers avec toutes les notes classées 1) par ordre alphabétiques, 2) pour nombre, 3) par durée, 4) en fichier CSV pour travail avec excel.

> *Note : cette option peut s'utiliser aussi avec l'application* `score-extract` *(**ScoreExtraction**) avec les mêmes options.*
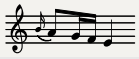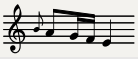
Remarques sur les statistiques

Les statistiques relèvent donc la fréquence d'utilisation de chaque note de la partition, ainsi que leur durée d'utilisation. Elles sortent le nombre de notes utilisées ainsi que la durée totale de chaque note mise bout-à-bout.

Certains partis-pris ont été adoptés :

- toutes **les répétitions** sont prises en compte (à l'avenir, si c'est nécessaire, on pourra imaginer une option qui permette de ne pas les prendre en compte)
- pour **les ornements**, on ne compte que la note elle-même (sauf pour les « grace notes » — cf. ci-dessous). En effet, comment considérer une trille par exemple ? Elle devrait comporter deux notes (les deux notes utilisées pour triller) et un certain nombre d'itérations indéfinissable de façon stricte avec des durées tout aussi indéfinissables. On pourrait se retrouver aussi avec des statistiques faussées qui amplifieraient l'utilisation d'une note simplement parce qu'elle est produite par la trille (on pourrait objecter que cette note n'est pas « amplifiée » puisqu'elle est, de fait, jouée dans la musique…).
- on fait une exception pour **les *grace notes*** (les ***petites notes***), donc, c'est-à-dire les notes explicitement écrites, avec une durée définie, qui seront prises en compte. Mais conformément à la tradition de jeu, pour l'appogiature « barrée » (petites notes barrées), on définit sa durée en fonction de la note qui la suit.

  Nous avons dû prendre certains partis au cause des écritures différentes selon les compositeurs. Par exemple, on trouver chez Mozart et chez Haydn deux écritures différentes pour le même effet :

MOZART	HAYDN	EFFET
		
Remarquer la petite note en double-croche et la liaison	*Remarquer la petite note en croche sans liaison*	*Mais les deux écriture produiront ce résultat avec deux double-croches.*

L'option prise est la suivante : la durée de la petite note, quelle que soit sa durée écrite, sera la moitié de celle de la note qui la suit. Dans les deux exemples ci-dessus, la petite note sera donc une double-croche, et la note suivante sera raccourcie d'autant, pour se rapprocher de l'effet produit.

> *Pour les petites notes supprimées, le calcul n'est pas encore optimisé et il peut souvent survenir des erreurs de calculs (certaines petites notes n'étant pas retirées de la partition analysée).*

Certaines erreurs découlent de l'écriture même et, pour le moment, ne peuvent pas être évitées. C'est le cas dans l'utilisation d'un arpège (ou similaire) conduisant à un accord, comme dans la partition suivante :



Dans cette partition, on comptera 2 Sol (celui en noire pointée et en croche lié à l'accord), 2 Do (celui en noire et celui en croche lié à l'accord).

Le Tempo

Le tempo (qui sert aussi pour jouer le fichier MIDI) doit toujours se mettre **en valeur de noire** (même si le tempo n'est pas à la noire). Si le tempo est écrit dans une autre valeur (blanche, croche…), alors faire la transposition (diviser par deux si c'est en croche, multiplier par deux si c'est en blanche).

On ajoute "**T**" au tempo lorsque c'est un rythtme ternaire (on met alors en tempo la valeur de la **noire pointée**).

Par exemple, pour une 6/8 où la noire pointée (le temps) devra être joué à 100, on indiquera :

```
--tempo 100T
```

Pour une mesure à 3/8 où les temps (donc la croche) doit être jouée à 100, on indiquera :

```
--tempo 33T
```

Puisqu'une noire pointée, ici durera trois fois plus de temps qu'une croche.

Options principales

Toutes les options dont nous allons parler peuvent être utilisées au début du code ou à n'importe quel endroit du fichier pour être modifiées à la volée. Par exemple, si on veut que les premières images soient produites sans barres de mesures, on ajoutera en haut du fichier l'option **`--barres OFF`** et si à partir d'une image on en veut, on pourra poser `--barres` et remettre plus loin `--barres OFF` pour spécifier qu'il faut à nouveau ne plus utiliser les barres de mesure.

Résumé des options

> *Pour désactiver une option (après l'avoir activée ou pas), il faut utiliser :*
> *`--<option> OFF`*
>
> *Par exemple :*
>
> ***`--barres OFF`***
>
> *... pour ne plus afficher les barres de mesure pour toutes les images suivantes dans le fichier mus.*

EFFET RECHERCHÉ	OPTION	NOTES
Tonalité de la pièce	**`--key <tune>`** **`--tune <tune>`**	Se définit par le nom de la note (a-z) en majuscule ou minuscule, l'altération et le mode. Cf.

		Tonalité de la pièce
Suppression de la gravure des barres	`--barres OFF`	
Ré-affichage des barres de mesure	`--barres`	S'emploie forcément après un `--barres OFF`, puisque par défaut les barres sont toujours gravées.

Afficher (ou non) la métrique	`--time`
	`--time OFF`
	`--time 3/4`	

Ne traiter que les images inexistantes	`--only_new`	Dans le cas contraire, toutes les images seront toujours traitées, qu'elles existent ou non, ce qui peut être très consommateur en énergie.
Ne pas afficher les hampes des notes	`--no_stem`	
Transposition du fragment	`--transpose <from> <to>`	Par exemple, `--transpose bes c'` va transposer le fragment, qui est en SI bémol, en Do, en prenant les notes les plus proches. Cf. Transposition
Taille de la page	`--page <format>`	Par défaut, la partition s'affiche sur une page A0 en format paysage, ce qui permet d'avoir une très longue portée. `<format>` peut avoir des valeurs comme `a4`, `b2` etc.
Espace vertical entre les portées	`--staves_vspace <x>`	Pour avoir l''espace normal, mettre 9. Au-delà (11, 12 etc.) on obtient un écart plus grand que la normale. "Staves vspaces" signifie (espace vertical entre les portées)
Espace vertical entre les systèmes	`--systems_vspace`	
Produire le fichier MIDI	`--midi`	Régler la valeur de tempo ci-

		dessous pour avoir le tempo.
Ouvrir le fichier image après production	**`--open`**	Ouvre tout de suite le fichier dans Affinity Designer, ce qui permet de l'améliorer si nécessaire.
Conserver le fichier LilyPond (`.ly`)	**`--keep`**	Cela permet de tester du code ou de voir où se situe un problème compliqué.
Détail des erreurs	**`--verbose`**	Permet de donner les messages d'erreur dans leur intégralité et notamment avec leur backtrace.
Portées multiples (cf. ci-dessous)	**`--staves_keys G,A,…`** **`--staves_names 1re,2e…`**	Permet de produire des portées empilées avec les clés et les noms voulus.
Nommage de la portée	**`--staves_names <nom>`**	Permet, notamment pour le piano, de préciser qu'il faut indiquer le nom (simplement en indiquant `--staves_names Piano`)
Taille de la portée	**`--staff_size <x>`**	Définit la taille de la portée (et de tous ses éléments). La valeur par défaut est **20**.
Affichage des numéros de page	**`--page_numbers <v>`**	`<v>` peut-être `OFF` (pas de numéro de page), `arabic` (chiffres arabes, `roman-ij-lower` (romain minuscules avec ligature), `roman-ij-upper` (romain majuscule avec ligature), `roman-lower` (romain minuscule sans ligature), `roman-upper` (romain majuscule sans ligature)
Statistiques	**`--stats`**	Produit toujours les statistiques en même temps que la partition, dans un dossier `stats`.
Tempo pour les	**`--tempo <valeur>`**	Ne sert que pour les statistiques

statistisques et le fichier MIDI		et le fichier MIDI. Si on doit ajouter l'indication « noire = valeur » au-dessus de la première portée, il ne faut pas le faire avec *ScoreImage*.
Arrêt de la fusion des silences	`--merge_rests OFF`	Cf. Fusion des silences.
Suppression des doigtés	`--no_fingers`	Permet de supprimer l'écriture des doigtés.

Détail des options

Tonalité de la pièce

L'option `--tune` ou `--key` permet de définir l'armure ou la tonalité de la pièce.

La forme canonique est :

```
<note><altération><mode>

où :

  <note>  est une lettre de a à g (ou A à G)
  <altération> est 'es' ou 'b' pour bémol, 'is' ou '#' ou 'd' pour dièse
  <mode> est 'm' ou '-' pour le mineur (rien pour le majeur
```

Voilà différentes formes valides de définitions :

- `--tune a` = tonalité de La majeur
- `--key Gm` = tonalité de Sol mineur
- `--tune bes-` = tonalité de Si bémol mineur

- `--tune c-` = tonalité de Do mineur

Numérotation des pages

On règle la numérotation des pages avec l'option **`page_numbers`**

```
--page_numbers OFF              # pas de numérotation
--page_numbers arabic           # chiffres arabes (défaut)
--page_numbers roman-lower      # Romains minuscules
--page_numbers roman-upper      # Romains majuscules
--page_numbers roman-ij-lower   # Romains minuscules avec ligatures (*)
--page_numbers roman-ij-upper   # Romains majuscules avec ligatures (*)

(*) Semble ne pas fonctionner, peut-être faut-il une police spéciale.
```

Portées multiples

On définit les portées multiples à l'aide de **`--staves_keys`** (pour les clés) et **`--staves_names`** (pour les noms — voir aussi Nommage des portées).

On doit les définir **de bas en haut**. C'est-à-dire que si on veut un violon au-dessus d'un piano, on doit définir :

```
--staves_keys F,G,G
--staves_names Piano,Piano,Violon
```

Le simple fait qu'on trouve deux fois de suite le mot « piano » indique à ScoreImage de relier les deux portées.

> *Les noms des instruments doivent être mis en capitales si on veut qu'ils soient en capitales sur la partition.*

En fait, ci-dessus, la marque « Piano,Piano » (qui pourrait être aussi « PIANO,PIANO » indique à ImageScore qu'on a une portée de piano. Il produit alors deux portées reliées dans un système propre au piano, avec une accolade, et une portée de violon. On l'appelle une « sonate avec piano » (sonate with piano).

Espacement entre les systèmes

L'espace vertical entre les systèmes se définit à l'aide de l'option **`systems_vspace`**.

Par exemple :

```
--systems_vspace 30
```

Pour l'espacement vertical entre les portées d'un système, cf. ci-dessous.

Espacement entre les portées

L'espace vertical entre les portées se définit à l'aide de l'option **`staves_vspace`**.

Par exemple :

```
--staves_vspace 40
```

Pour l'espacement vertical entre les systèmes, cf. Espacement entre les systèmes.

Taille des portées

La taille des portées se règle à l'aide de l'option **`--staff_size`** suivie de la valeur à lui donner. La valeur par défaut est 20.

```
--staff_size 22.5
```

Numérotation des mesures

Par défaut, LilyPond et donc *ScoreImage* numérote les mesures, au début de chaque système.

Pour supprimer toute numération, utiliser l'option :

```
--mesure OFF
```

Pour forcer la numérotation de la première mesure, utiliser :

```
--first_measure
```

> *Noter que c'est nécessaire seulement si la première mesure est la 1. Dans le cas contraire, on numérote toujours la première.*

Pour partir d'une autre mesure que 1, utiliser :

```
-> mon_image
--mesure 12
c d e f
```

La numérotation pour le code ci-dessus commencera à partir de la mesure 12.

Pour que les numéros se fassent **de 5 en 5** ajouter :

```
--number_per_5
```

Ou par un autre numéro, quelconque :

```
--number_per 12
# => numérotation des mesures de 12 en 12
```

Par défaut, les numéros de mesure se mettent au-dessus de la portée. Pour les mettre en dessous, utiliser l'option :

```
--measure_number_under_staff
```

Nommage des portées

On peut nommer les portées à l'aide de l'option `--staves_names` (cf. Portées multiples).
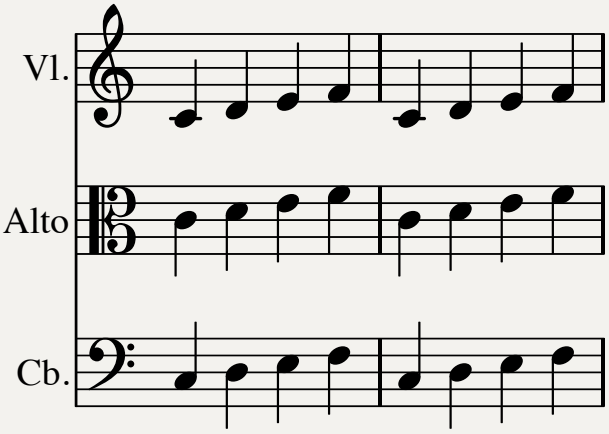
On peut ajouter un vrai dièse ou un vrai bémol dans le nom en utilisant `_b_` pour le bémol et `_d_` ou `_#_` pour le dièse. Ils seront remplacés par de vrais signes bémols et dièse, pour un affichage parfait.

Groupement des portées

Il existe plusieurs façons de relier les portées et les barres de mesure. On trouve les valeurs suivantes :

- Portées
    - non reliées
    - reliées par un trait simple (type neutre)
    - reliées par un trait + crochet oblique (type quatuor, chœur)
    - reliées par un trait + accolade (type piano)
- Barres de mesure
    - reliées entre elles
    - non reliées entre elles
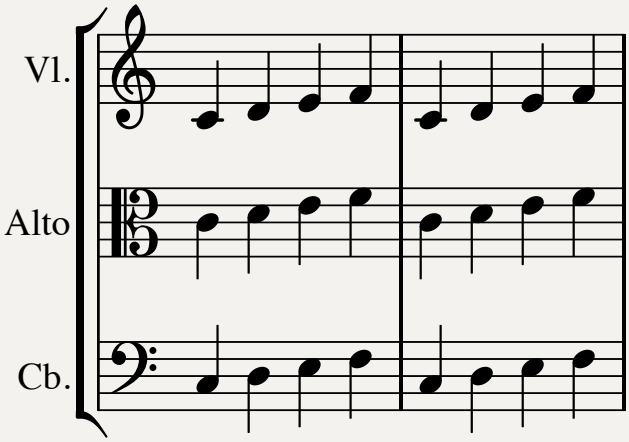
Par défaut, on utilise : **portées reliées par un trait simple avec les barres de mesure non reliées**.



Quand on veut un **crochet oblique**, on utilise « `[...]` ». Par exemple :

```
--staves_names [Cb., Alto, Vl.]
--staves_keys  F, UT3, G
```
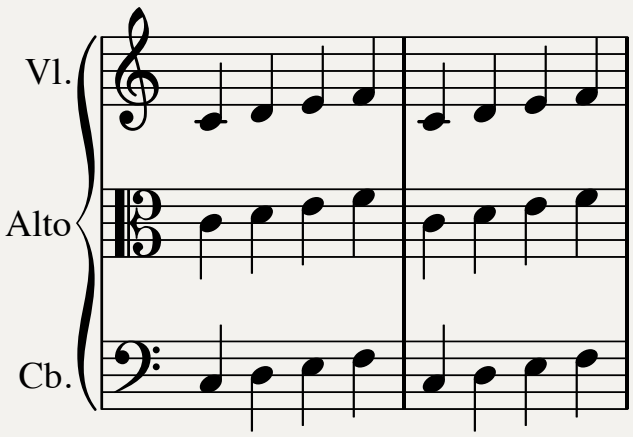
… produira :



Quand on veut une accolade, on utilise « `{...}` ». Par exemple :

```
--staves_names {Cb. Alto., Vl.}
--staves_keys  F, UT3, G
```

… produira :
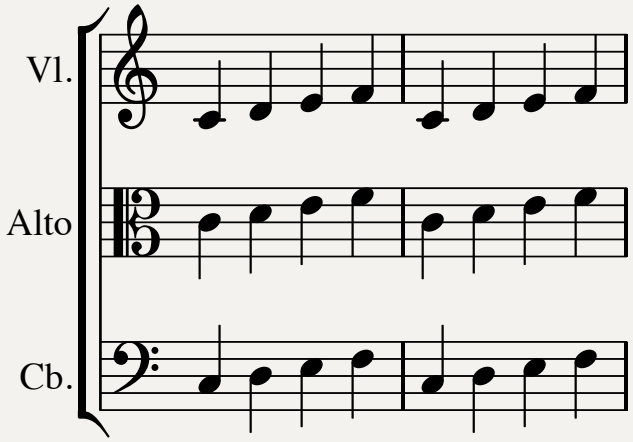


Comme on peut le voir, par défaut, les barres de mesure sont reliées entre elles. Pour utiliser **les barres de mesure non reliées**, on ajoute un « `-` » après l'accolade ou le crochet.

Par exemple :

```
--staves_names [-Cb., Alto, Vl.]
--staves_keys  F, UT3, G
```

… produira :



Tandis que :

```
--staves_names {-Cb. Alto., Vl.}
--staves_keys  F, UT3, G
```
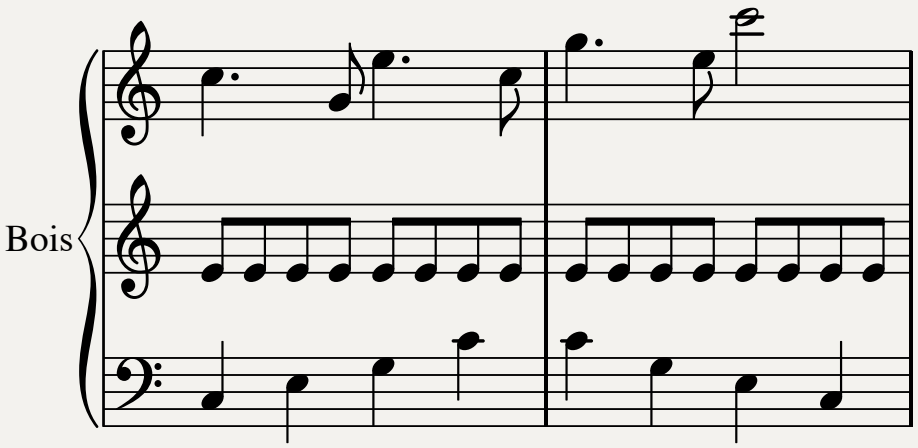
… produira la même chose que les accolades seules, puisqu'**un groupement de ce type relie toujours ses barres de mesure**.

Si tous les instruments du groupe portent le même nom, il est utilisé comme **nom du groupe** et les portées ne sont plus nommées individuellement.

Comme dans :

```
--staves_names {Bois, Bois, Bois}
--staves_keys F, G, G
```

... qui produira :



Groupes à l'intérieur d'un même système

On peut utiliser de la même manière les regroupements de portées à l'intérieur même d'un groupe, avec les accolades et les crochets, en indiquant par un « moins » l'absence de barres de mesure reliées.

Par exemple :

```
--staves_names Cb. {Piano, Piano}, Vl.
--staves_keys  F, F, G, G
```

... produira :

Cas spécial du PIANO

Le cas du piano est spécial car il possède sa propre option :

```
--piano
```

C'est une écriture simplifiée pour :

```
--staves_names {Piano, Piano}
--staves_keys F, G
```

Mais plus encore, ça simplifie l'écriture quand il y a des variables, puisque dans la définition de la partition on n'est pas contraint de préciser les deux mains :

```
--barres
--piano


mesure1=
c'4 d e f
c,1

mesure2=
g a b c
g1

-> score
mesure1 mesure2
```

On pourra cependant utiliser l'écriture normale :

```
--barres
--piano
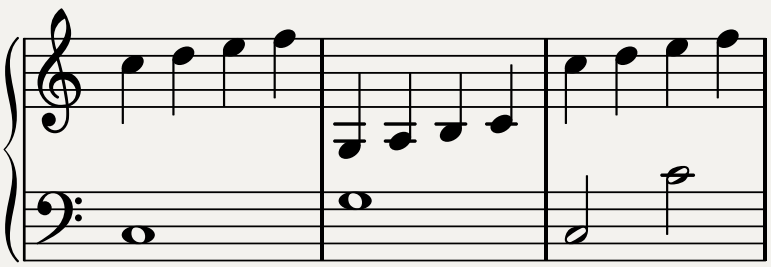

mesure1=
c'4 d e f
c,1
```

```
8
9  mesure2=
10 g a b c
11 g1
12
13 -> score
14 mesure1 mesure2
15 mesure1 mesure2
```

… notamment dans le cas d’une utilisation d’autres variables. Par exemple :

```
1  --barres
2  --piano
3
4  mesure1=
5  c'4 d e f
6  c,1
7
8  mesure2=
9  g a b c
10 g1
11
12 mesure3=
13 r1
14 c,2 c'
15
16 -> score
17 mesure1 mesure2 mesure1
18 mesure1 mesure2 mesure3
```
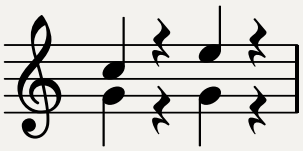
… qui produira :
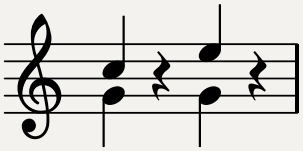


> *Noter ci-dessus que c'est seulement la main gauche de la mesure 3 qui a été utilisé, alors que la main droite a été empruntée à la mesure 1, conformément à la définition de la partition.*

Fusion (merge) des silences

Par défaut, *ScoreImage* fusionne les silences (ce que ne fait pas LilyPond). Cela signifie qu'au lieu de graver :



… *ScoreImage* gravera :



On peut désactiver ce comportement par défaut en utilisant l'option `--merge_rests OFF` (note : « merge » signifie « fusionner » et « rest » signifie « repos », « silence » en musique).

```
--merge_rests OFF

-> score
<< \stemDown g' r g r // \stemUp c r e r >>
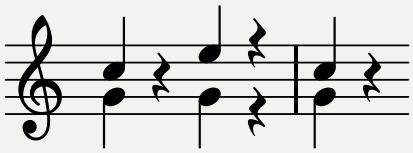
# => Les silences seront conservés comme ci-dessus
```

Arrêt ponctuel de la fusion des silences

Pour le moment, ça ne fonctionne pas dans toutes les situation. Si c'est le cas, utiliser le positionnement précis du silence p.e. `a'8\rest`.

Si l'on conserve le comportement par défaut, on peut néanmoins désactiver localement la fusion des silences grâce à la marque `\not_merge_rests` (et la faire reprendre à l'aide de `\merge_rests`) :

```
<< { \stemDown g' r } \\ { \stemUp c r } >> \not_merge_rests << {
\stemDown g r } \\ { \stemUp e' r } >> \merge_rests << {
\stemDown g, r } \\ { \stemUp c r } >>
```
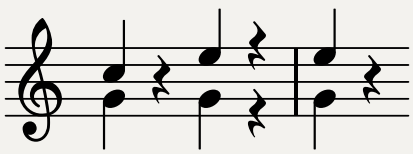


Il peut être plus simple et plus lisible de fonctionner avec variable. Par exemple, pour produire le code ci-dessus (à quelques variations près, ajoutées pour bien montrer le traitement différent de la même variable) :

```
b1=
<< \stemDown g' r // \stemUp c r >>

b2=
<< \stemDown g' r // \stemUp e' r >>

-> score
b1 \not_merge_rests b2 \merge_rests b2
```

Ce code produira :



Notez plusieurs choses importantes ici :

- La marque `\not_merge_rests` s'applique à un « contexte », c'est-à-dire à un

bout de code musical bien défini. On ne peut pas l'insérer par exemple de cette manière :

```
<< \stemDown g' r g \not_merge_rests r // \stemUp c r e r >>
# => NE PRODUIRA PAS LE RÉSULTAT OBTENU
```

L'utilisation des variables, comme indiqué ci-dessus, permet de gérer ce problème, puisqu'une variable est forcément « isolée » dans le code LilyPond produit.

- La marque `\not_merge_rests` est *définitive*. C'est-à-dire que tant que le *ScoreImage* ne rencontre pas de `\merge_rests`, les silences ne seront plus fusionnés.

Fichier de sortie MIDI

Grâce à l'option **`--midi`** dans le code mus on peut produire un fichier MIDI à écouter (dans VLC par exemple).

Cette option marche de paire avec l'option **`--tempo`** qui détermine le tempo de l'écoute.

Par exemple :

```
# ./score.mus

--midi
--tempo 120

c d e f
```

… produira un fichier `./score/score.midi` qu'on pourra écouter dans VLC et qui jouera les notes au rythme de 120 à la noire.

À voir aussi : le tempo.

Pour produire le fichier MIDI seul l'option `--midi` dans le code MUS ne suffit pas. Il faut l'utiliser dans la ligne de commande. Par exemple :

```
score-image moncode.mus -midi
```

> *Bien noter que la commande ci-dessus ne produira QUE le fichier midi (à partir du code du fichier « moncode.mus »). Ce qui est très pratique pour les très longues partitions qui demandent plusieurs dizaines de secondes en gravure des partitions.*

Si un tempo est ajouté à la ligne de commande, il supplante le tempo défini dans le fichier .mus :
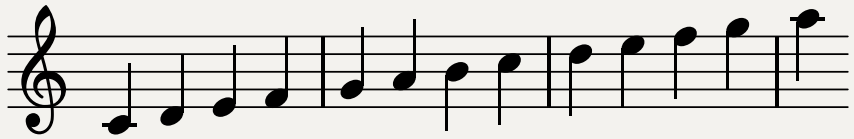
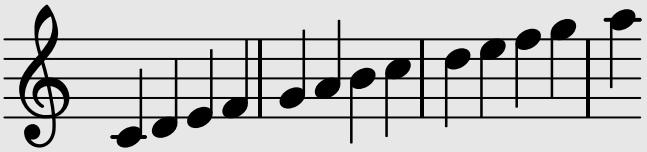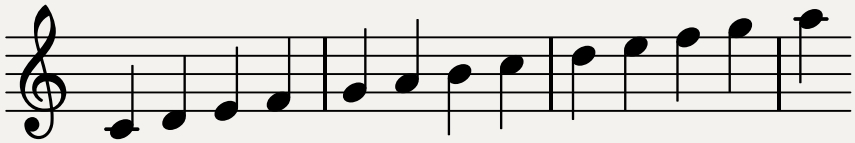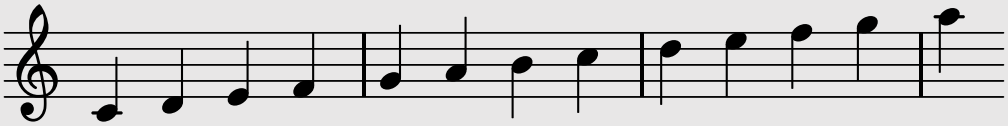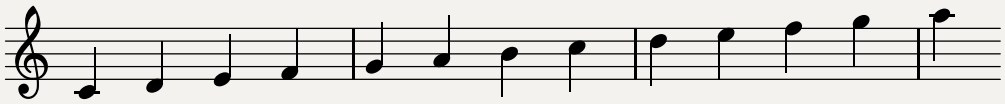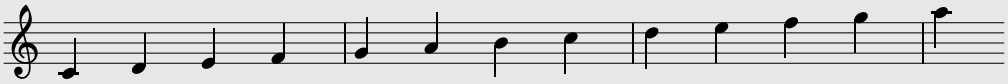
```
score-image moncode.mus --midi --tempo=80
```

Cela permet de produire très rapidement des fichiers MIDI plus lents (pour contrôle des notes) ou plus rapides (pour le fun).

> *Concernant le traitement des petites notes dans les fichiers MIDI, qui peuvent poser de gros problèmes, voir le chapitre Suppression des petites notes.*

Espacement entre les notes (`proximity`)

On peut jouer sur la proximité entre les notes à l'aide de l'option **`--proximity`** suivie de la valeur d'éloignement (ou de rapprochement, c'est selon).

VALEUR DE `PROXIMITY`	RENDU
Non définie	 *(correspond ici à la proximité 5)*
`--proximity 1`	
`--proximity 5`	
`--proximity 10`	
`--proximity 20`	
`--proximity 50`	

Cette option s’applique à toutes les images suivantes dans le fichier MUS, mais peut être changée en cours de processus. Par exemple, le code :

```
--proximity 2

notes=
g' f e d c1

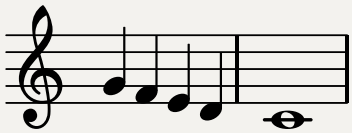
-> score_prox2
notes

--proximity 50
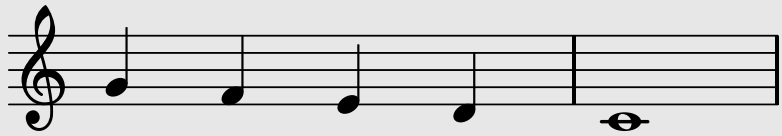
-> score_prox50
notes

```

... produira les images :



score_prox2.svg



score_prox50.svg

Commandes

Le code `MUS` répond à quelques commandes permettant de jouer sur la compilation.

Ces commandes sont reconnaissables au fait qu'elles sont toujours en capitales.

Commande de déclenchement de la compilation (START/STOP)

Lorsqu'un fichier est volumineux et contient de nombreuses images, on peut vouloir se concentrer parfois seulement sur quelques unes d'entre elles, sans avoir à les produire toutes à chaque fois, ou sans avoir à détruire celles qu'on veut revenir quand on utilise l'option `only_new`.

On utilise alors la commande **`START`** pour indiquer le début du travail et la commande **`STOP`** pour indiquer où le fichier.

Par exemple, le fichier :

```
# in durees.mus

-> image1
c8 d e f

START

var=
```

```
c4 d e f

-> image2
var

STOP

-> image3
c2 d e f
```

... ne produira que l'image « `durees/image2.svg` » .

> *Noter que la commande se trouve sur une lignes « isolée », conformément au principe des paragraphes.*
>
> *Noter qu'il faut inclure la définition des variables dans le bloc « START ... STOP ». Dans le cas contraire, une erreur de variable inconnue sera produite.*

Nom du fichier de l'image (définition explicite)

Si une ligne commençant par `->` est placée avant l'expression musicale, elle contiendra le nom du fichier de sortie.

Par exemple :

```
-> monfichier
c d e f
```

... placera dans le fichier //monfichier.svg la partition
résultant de l'expression `c d e f`.

Notation LilyPond simplifiée
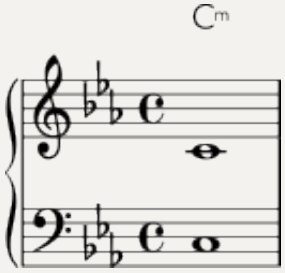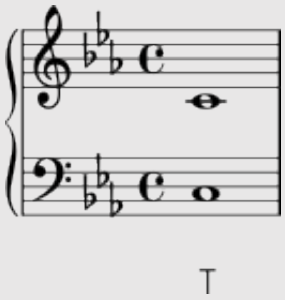
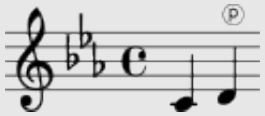
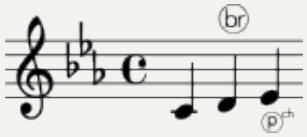
Cette section présente les notations de l'expression pseudo-lilypond qui diffèrent du langage original (toujours pour simplifier).

Chiffrage/harmonisation/Analyse mélodique

Depuis la version de septembre 2025, on peut utiliser la librairie `chiffrage.ly` qui offre des fonctions pour marquer les accords et les notes.

Pour l'utiliser, ajouter l'option `--chiffrage`.

Puis utiliser les fonctions :

FONCTION	DESCRIPTION	APERÇU
`\chifName "<...>"`	Pour le nom de l'accord au-dessus de la portée. `c\chifName "Cm"`	
`\chifDeg "<...>"`	Pour indiquer le degré ou la fonction de l'accord sous la portée. Par exemple `c\chifDeg "T"` va produire :	
`\chifNote <type> <pos>`	Pour indiquer une note particulière, de passage, broderie, double broderie, etc. Voir les exemples ci-dessous.	
	`c d\chifNote "np" #up` ou `c d\chifNP`	
	`c d\chifNote "br" #up e\chifNote "npc" #down`	
	On peut utiliser de la même manière toute les marques de la police PhilNote (version 6 à l'heure où nous écrivons ces lignes).	
	TODO Faire toutes les fonctions raccourcis `\chifBR` (pour « broderie ») etc.	

Répétitions
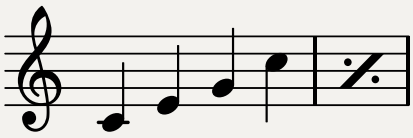
Marque de répétition avec le signe pourcentage

Forme canonique : `{ <contenu> }x<nombre de fois>`

Exemples :

```
{ c e g c }x2
```
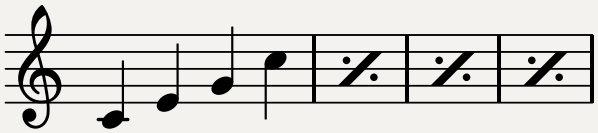
Produira :



On peut indiquer des répétition de 2 à 4 :

```
{ c e g c }x4
```

Produira :



Répétition d'une note avec `<note>*N`

De la même manière qu'on peut faire `r8*4` en pur Lilypond, on peut répéter n'importe quelle note avec durée à l'aide de l'astérisque. Par exemple :

```
cis16.*3
```

… produira le code

```
cis16. cis16. cis16.
```

> *Noter que pour éviter toute confusion, cette possibilité se limite strictement à des notes avec ou sans altérations et pouvant définir leur durée. Tout autre groupe — par exemple présentant le doigté ou l'articulation — sera ignoré. Il faut alors utiliser d'autres moyens de répétitions (cf. ci-dessous).*

Si une octave est appliquée à la note répétée, elle ne sera appliquée qu'au premier item.

Ainsi, le code :

```
cis'16*4
```

… produira :

```
cis'16. cis16. cis16. cis16.
```

Répétition d'un code avec `% ... %N`

On peut utiliser le gabarit :

```
% ... %N
```

… pour répéter un nombre illimité de fois un motif.

Par exemple, pour répéter 8 fois la séquence **`c8 d e`**, il suffit de faire :

```
% c8 d e %8
```

Si le segment répété doit se trouver à une octave particulière, on peut enrouler le code dans un **`relative`** :

```
\relative c,, { % c8 d e %8 }
```
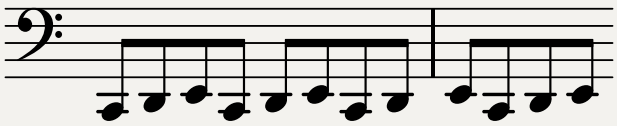
Mais on peut le faire plus simplement avec :

```
\cle F % c,,8 d e %4
```

… car l'octave de départ sera supprimée pour donner le code :

```
\cle F c,,8 d e c d e c d e c d e
```

… qui produira :



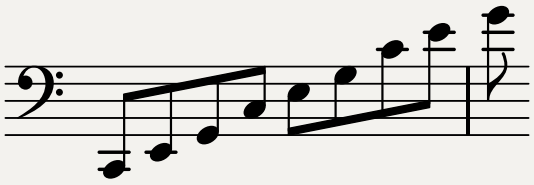
Il est important, néanmoins, de surveiller l'octave à la fin de la chaine, car si on fait :

```
\cle F % c,,8 e g %3
```

… on produira le code :

```
\cle F c,,8 e g c e g c e g
```
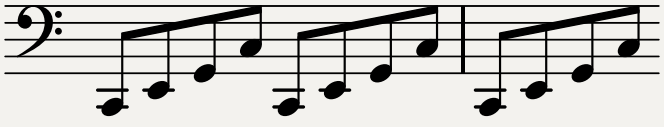
… qui verra monter sans redescendre l'arpège de Do majeur, car le deuxième Do repartira à l'octave du Sol précédent, etc. :



Dans ce cas, on peut utiliser les **marques d'octaves absolues** qui permettent de toujours repartir sur la même note en début de répétition :

```
\cle F % c=,8 e g c %3
```
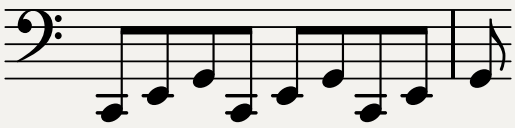
… qui produira :



Dans les cas inextricables, on peut utiliser avec plus de sécurité le code normal de Lilypond :

```
\cle F \repeat unfold 3 { c,,8 e g }
```

… qui produira :



##

Cette section présente les notations de l'expression pseudo-lilypond qui diffèrent du langage original (toujours pour simplifier).

Les Barres

OBJET	CODE	DESCRIPTION
Début de reprise	`|:`	
Fin de reprise	`:|`	
Fin et début de reprise	`:|:`	
(*Code Lilypond pour les autres barres*)		
Fin de pièce	`|.`	
Séparation de partie	||	

1re, 2e, etc. fois dans les reprises

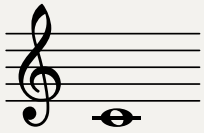
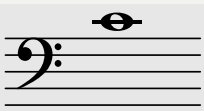
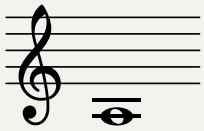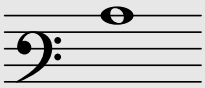
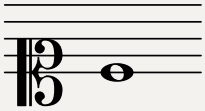
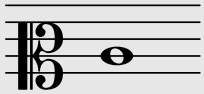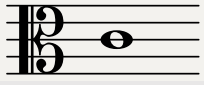
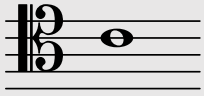
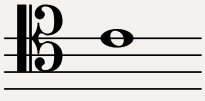
Les premières, deuxième, etc. fois se gèrent à l'aide `|<x>` où `<x>` est le numéro de l'alternative :

```
|:  .... |1 ... |2 ... |3,4 ... :|5 ... |6,7 || suite
```

> *Note 1 :*
>
> *La barre « `||` » délimitant la dernière fois peut être aussi une autre reprise « `|:` » ou une barre de fin « `|.` ».*
>
> *Note 2 :*
>
> *Il peut ne pas y avoir de barre de reprise de début, lorsqu'on revient au début pour faire la reprise.*

Clé de l'expression

On peut utiliser les marques normale de LilyPond mais il peut être plus pratique d'utiliser :

OBJET	CODE	DESCRIPTION
	`\cle G`	Clé de SOL 2e ligne
	`\cle F`	Clé de FA 4e ligne
	`\cle G1`	Clé de SOL 1ère ligne
	`\cle F3`	Clé de FA 3e ligne
	`\cle UT1`	Clé d'UT 1ère ligne
	`\cle UT2`	Clé d'UT 2e ligne
	`\cle UT3`	Clé d'UT 3e ligne
	`\cle UT4`	Clé d'UT 4e ligne
	`\cle UT5`	Clé d'UT 5e ligne

Tonalité de l’expression (armure et changement)

Pour la définition de l’armure, voir Tonalité de la pièce.
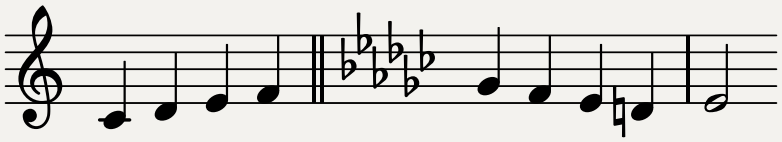
On marque un changement de tonalité au cours de la pièce simplement avec la marque `\tune` ou `\key`.

Pour ne pas ajouter la double barre, on écrit l’expression en « pur » LilyPond, c’est-à-dire avec la marque `\key` (pas `tune` cette fois), la note en minuscule et l’altération par `es` (bémol) ou `is` (dièse). On peut indiquer si c’est une tonalité mineure par `\minor` ou utiliser l’armure du relatif majeur.

Ainsi :

```
c d e f \tune Ebm ges f ees d ees2
```

…produira :



Tandis que :
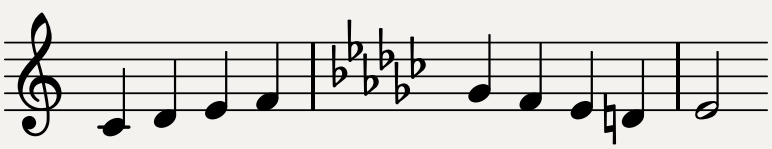
```
c d e f \key ees\minor ges f ees d ees2

# ou

c d e f \key ges ges f ees d ees2
```

> *Noter que le `\minor` est « collé » à la note.*

… produiront tous les deux :



Transposition
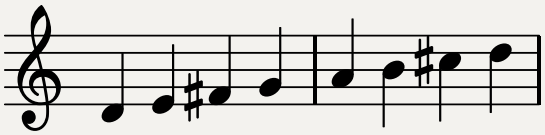
On peut transposer tout un fragment à l'aide de l'option `--transpose` en indiquant la note/tonalité de référence (aka la note de départ) puis la note/tonalité d'arrivée.

Par exemple :

```
--transpose c d

-> score
c d e f g a b c
```

… transposera le fragment de Do majeur vers Ré majeur :
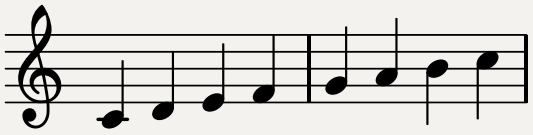


Tandis que :

```
--transpose d c

-> score
d e fis g a b cis d
```

… transposera ce fragment en Ré majeur de Ré majeur vers Do majeur :
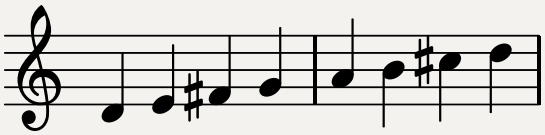


NB: Si vous voulez que l'armure tienne compte de la transposition quand la tonalité est Do majeur, il faut écrire explicitement cette tonalité à l'aide de l'option `--tune c`.

Ainsi, le code :

```
--transpose c d

-> score
c d e f g a b c
```

… produira :
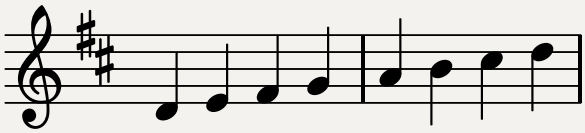


Tandis que le code :

```
--tune c
--transpose c d

-> score
c d e f g a b c
```

… produira lui :



> *Si la tonalité de départ est auteur de Do majeur, elle sera indiquée donc elle sera traitée de la même manière.*

Instruments transpositeurs

Le traitement des instruments transpositeurs est extrêmement simple avec *music-score* : il suffit d'indiquer `\trans` suivi de la note de transposition (telle quelle) pour produire les bonnes notes avec la bonne armure.

Par exemple, pour une clarinette en La (A), il suffit d'indiquer `\trans a` :

```
-> score
\trans a { c d e f }
```

Exemple avec un saxophone en Eb, un cor anglais (qui est en F), une clarinette en Sib, une flûte :

```
--barres
--staves_names Sax Eb, Cor A., Cl.(Bb), Fl.
--staves_keys  G, G

fl=
e'1

cl=
c4 d e f

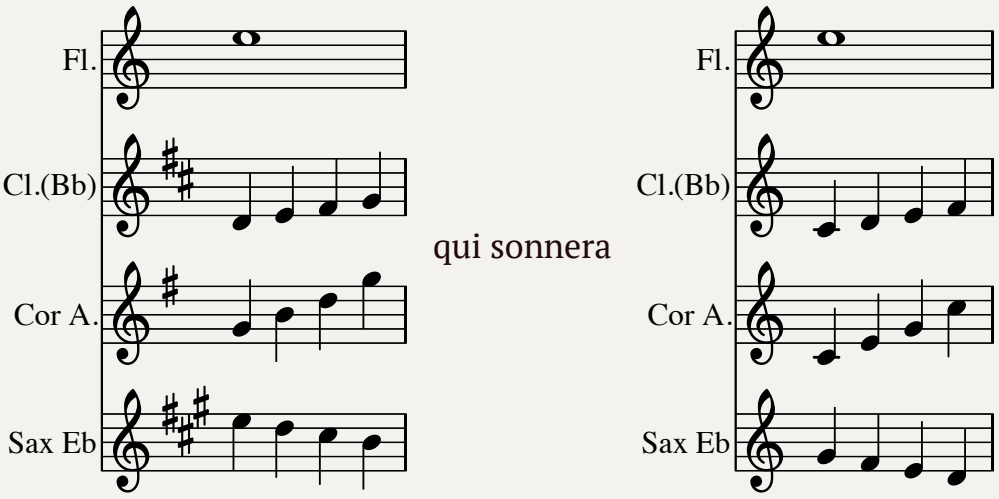
cor_anglais=
c4 e g c

sax=
g'4 f e d

-> score
fl
\trans bes { cl }
\trans f { cor_anglais }
\trans ees { sax }
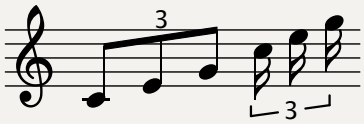
```

... produira la partition :



Triolet, quintolet et septolet
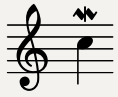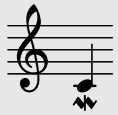
On les notes `3{note<duree> note note}`
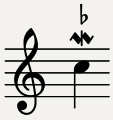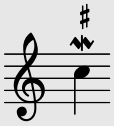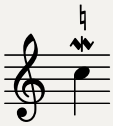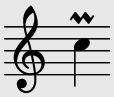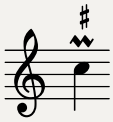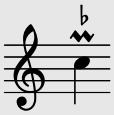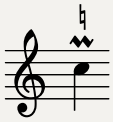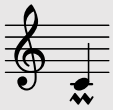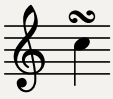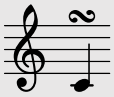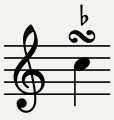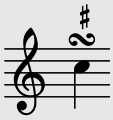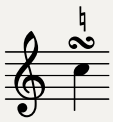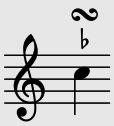
OBJET	CODE	DESCRIPTION
	`3{note<duree> note note}`	TODO : il faudra traiter les duolets, quintuplets et autres sextolets de la même façon.
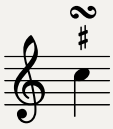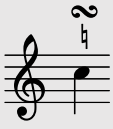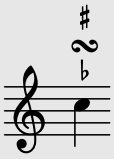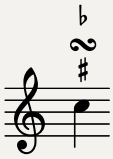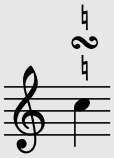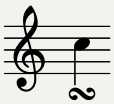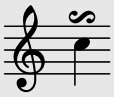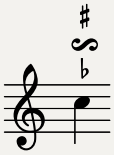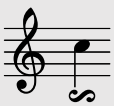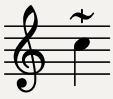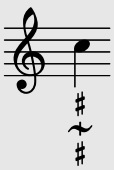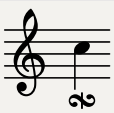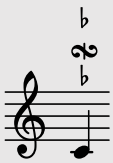
Pour s'assurer que le rythme soit bon (et que les statistiques soient bien calculées), ne pas oublier de mettre la durée sur la note ou le groupe de notes suivant cette marque. Par exemple :

```
3{c8 d e} f8
# => pour que le Fa suivant soit bien compté en croche simple.
```

Ornements et signes d'interprétation

OBJET	CODE	DESCRIPTION
	`c'\mordent`	Mordant inférieur (au-dessus de la note)
	`c_\mordent`	Mordant inférieur sous la note
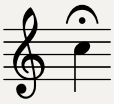
	`c'\mordentb`	Mordant inférieur avec note bémolisée (au-dessus de la note) Le bémol est le « b » après « mordent »
	`c'\mordent#`	Mordant inférieur avec note diésée (au-dessus de la note) Le dièse est le « # » après « mordent »
	`c'\mordentn`	Mordant inférieur avec note bécarisée (au-dessus de la note) Le bécarre est le « n » après « mordent » (« n » pour « natural »)
	`c'\prall`	Mordant supérieur (au-dessus)
	`c'\prall#`	Mordant supérieur diésé
	`c'\prallb`	Mordant supérieur bémolisé
	`c'\pralln`	Mordant supérieur bécarisé
	`c_\prall`	Mordant supérieur sous la note
	`c'\turn`	Gruppetto sur la note.
	`c^\turn`	Pour forcer la gravure du gruppetto au-dessus de la note, on peut ajouter `^` conformément au langage LilyPond. Noter que dans l'exemple, c'est superflu.
	`c'\turnb`	Gruppetto sur la note avec note bémolée au-dessus
	`c'\turn#`	Gruppetto sur la note avec note diésée au-dessus
	`c'\turnn`	Gruppetto sur la note avec note bécarrisée au-dessus
	`c'\turn/b`	Gruppetto sur la note avec note bémolée au-dessous

	`c'\turn/#`	Gruppetto sur la note avec note diésée au-dessous
	`c'\turn/n`	Gruppetto sur la note avec note bécarrisée au-dessous
		Toutes les autres combinaisons sont possibles. Par exemple :
	`c'\turn#/b`	Gruppetto sur la note avec note diésée au-dessus et note bécarrisée au-dessous
	`c'\turnb/#`	Gruppetto sur la note avec note bémolée au-dessus et note diésées en dessous
	`c'\turnn/n`	Gruppetto sur la note avec note bécarrisée au-dessous et au-dessus
	`c'_\turn`	Gruppetto sous la note.
	`c'\reverseturn`	Gruppetto inversé
	`c'\reverseturn#/b`	Gruppetto inversé avec notes altérées
	`c'_\reverseturn`	Gruppetto inversé sous la note
	`c'\haydnturn`	Gruppetto haydnien
	`c'\haydnturn#/#`	Gruppetto haydnien en dessous avec notes altérées
	`c'_\slashturn`	Gruppetto barré en dessous
	`c^\slashturnb/b`	Gruppetto barré avec notes altérées
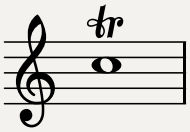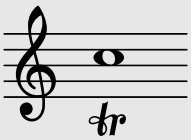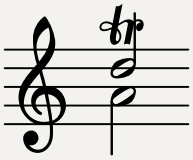
Pour d’autres ornements, voir https://lilypond.org/doc/v2.21/Documentation/notation/list-of-articulations.
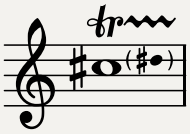
Voir aussi Marques d’expression.

Signes d’interprétation

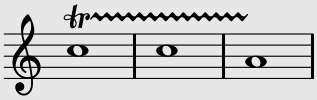
OBJET	CODE	DESCRIPTION
		`c'\fermata`
		`c_\fermata`

Trilles

OBJET	CODE	DESCRIPTION
	`\tr(c')`	Noter la note trillée entre parenthèses.
	`\_tr(c1')`	Pour forcer la trille en dessous
	`<< d'2 // \^tr(a) >>`	Pour forcer la trille au-dessus. Bien sûr, c’est juste pour forcer la trille à s’afficher par défaut en dessous (et donc voir l’effet du circonflexe) que l’exemple ci-contre a été donné. Il n’est pas utilisable dans la réalité, puisqu’on penserait que c’est le Ré qui est trillé.
	`\tr(aes8.) ( g32 aes)`	Noter la parenthèse qui commence la liaison sur la note triée qui est “détachée” de la trille. Sinon la trille serait mal interprétée
	`\tr(cis'1 dis)`	Pour triller avec une autre note que la note
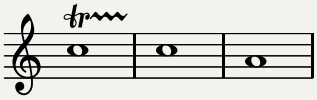


naturelle.



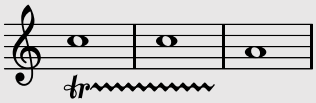
`\tr(c'1)- c a\-tr`

Noter le “tr-” pour commencer et le “-tr” pour finir. Le programme signalera une erreur si on oublie la marque de fin.
Noter que pour que la « vague » de la trille aille jusqu’à la note suivante, il faut introduire cette note dans la marque `\tr .... \-tr` (ou utiliser le code un peu plus bas). Dans le cas contraire, on obtiendra l’image ci-dessous.



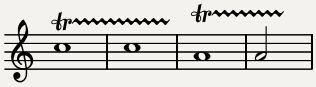
`\tr(c'1)- c\-tr a`

La « vague » de la trille s’arrête sur la dernière note comprise entre `\tr` et `\-tr`.
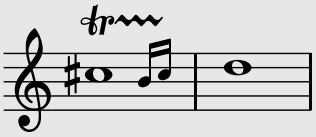


`\_tr(c'1)- c a\-tr`

La même chose en forçant la trille en dessous.



`c'1 \startTr c a \stopTr \startTr a2 s \stopTr`

Les codes ci-dessus ne permettant pas d’enchainer les trilles avec leur « vague », on utilise les marque `\startTr` et `\stopTr` comme ci-contre pour y parvenir.



`\tr(cis'1)- (b16 cis)\-tr d1`

Noter ici la tournure différente à la fin, avec les deux grâce-note entre parenthèses. Noter quand même la logique générale.
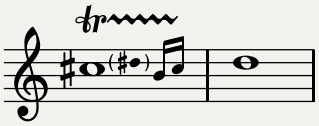


`\tr(a'1)-( (gis16 a)\-tr bes1)`

Même chose avec une liaison.



`\tr(a'1)- (gis16( a))\-tr bes1`
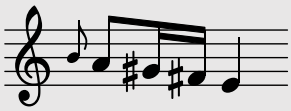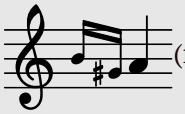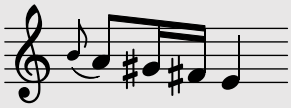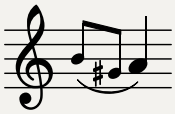
Même chose avec une liaison courte sur les deux petites notes



`\tr(cis'1 dis)- (b16 cis)\-tr d1`

On ajoute une note trillée avec une note étrangère

Petites notes (grace notes)

OBJET	CODE	DESCRIPTION
Non liées non barrées	`\gr(notes) note`	
Exemple simple	`\gr(b'16) a8 gis16 fais`	
Exemple multiple	`\gr(b'16 gis) a4`	
Non liées barrées	`\gr(note/)`	Remarquer la barre finale qui symbolise la note barrée
Exemple	`\gr(b'8/) a4`	
Exemple multiple	`\gr(b'16 gis/) a4`	 (noter : non barré)
Appoggiature	`\gr(note-)`	
Exemple	`\gr(b'8-) a gis16 fis e4`	
Exemple multiple	`\gr(b'8 gis-) a4`	
Acciaccature	`\gr(note/-) note`	
Exemple	`\gr(ais'16/-) b4`	
Quand plusieurs notes	`\grace note[ note note note]`	

> *Au niveau des statistiques, voir les notes concernant le traitements particulier des petites notes.*

Problèmes des petites notes dans les fichiers MIDI

Certaines *petites notes* peuvent poser problème dans les fichiers midi, en décalant toutes les notes d'une voix (lorsque les notes sont prises pour leur durée réelle). Si le fichier midi doit servir à produire la musique, il convient alors de supprimer les petites notes aux endroits problématiques et de les remplacer par leur vrai valeur.

S'il s'agit juste d'un fichier MIDI pour contrôler la partition par exemple, on peut utiliser les options `no_grace` pour spécifier les *petites notes* à supprimer (elles ne seront pas du tout jouées).

> *Dès à présent, précisons que ces options peuvent changer les octaves des notes et qu'il convient donc, par précaution, de préciser cette hauteur de façon explicite dans la note suivante, comme cela est expliqué dans la section Indication de l'octave exacte après une petite note.*

Voici le sens de ces options :

OPTION	DESCRIPTION	NOTE
`--no_grace`	Dans le code mus ou en ligne de commande, cette option supprime toutes les *petites notes*, quel que soit leur type.	
`--no_graces a`	Dans le code mus, cette option supprime toutes les appogiatures	En ligne de commande, ajouter « a » à `--no_graces=`.
`--no_graces c`	Dans le code mus, cette option supprime toutes les acciaccatura	En ligne de commande, ajouter « c » à `--no_graces=`.
`--no_graces g`	Dans le code mus, cette option supprime toutes les *grace notes*	En ligne de commande, ajouter « g » à `--no_graces=`.
`--no_graces s`	Dans le code mus, cette option supprime toutes les *petites notes* barrées	En ligne de commande, ajouter « s » à `--no_graces=`.
`--no_graces p`	Ne supprimer les *petites notes* que si elles sont entre parenthèses.	En ligne de commande, ajouter « p » à `--no_graces=` .

En ligne de commande, avec `--no_graces`, les lettres doivent être mises bout à bout. Par exemple :
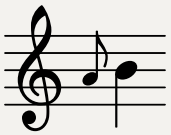
```
score-image --midi --no_graces=agp score.mus
# Supprime les  a: appoggiatures
#               g: grace notes
#               p: entre parenthèses
```

Indication de l'octave exacte après une petite note

Lorsque les *petites notes* sont supprimées (cf. ci-dessus) un problème d'octave peut se poser. Par exemple, le code :

```
\gr(a'8) b4
```

… produira de façon naturelle :
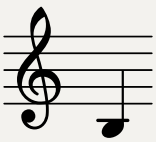


Mais si on lui ajoute l'option `--no_grace` qui supprimera toutes les *petites notes*, alors le code :

```
--no_grace

\gr(a'8) b4
```

… produira l'image :
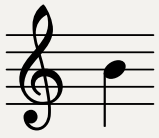


La note Si n'a plus été montée d'une octave par la *petite note* avant puisque cette *petite note* a été supprimée.

Pour palier ce problème, il suffit d'indiquer explicitement l'octave de la note à l'aide du signe « = » (égal). Et dans ce cas, le code :

```
--no_grace

\gr(a'8) b='4
```

… produira l'image :



Noter que pour l'octave médiane, il suffit de ne rien mettre après le signe égal :

```
--no_grace

b'' \gr(a8) b=4
```

Ce code produira :



Regroupement des hampes
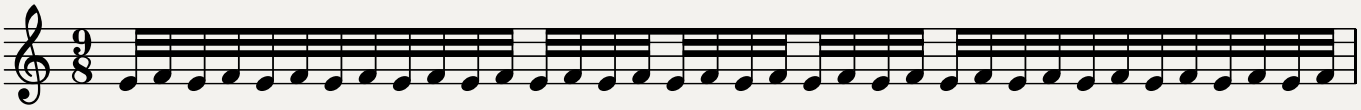
On peut forcer le regroupement des hampes dans certains cas grâce à la marque simplifiée **`\subdiv`** qu'on interrompt à l'aide de **`\subdivOFF`**.

Par exemple, le code :

```
--time 9/8

-> score
% e32 f %6 \subdiv % e32 f %6 \subdivOFF % e32 f %6
```

… produira :

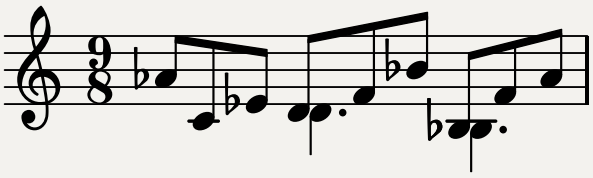


Notez le deuxième groupement de triples croches qui est affecté par la subdivision tandis que les autres ne le sont pas.
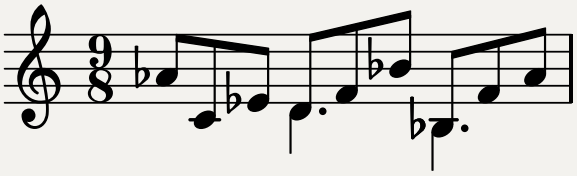
Voir aussi Attache des hampes.

Notes « mergées »

« Merger » des notes signifie utiliser une seule tête pour deux notes. Ce merge permet de transformer :



… en :



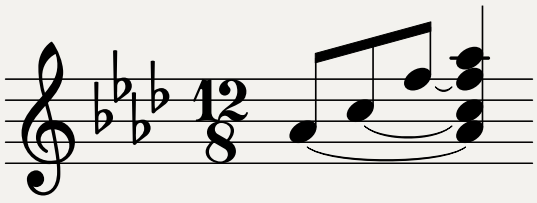
On obtient ce merge en ajouant `\mergeNotes` avant les notes. Ce merge sera effectif jusqu'à la prochaine parenthèse de délimitation de groupe de note (donc c'est variable en fonction de la façon d'écrire la musique avec LilyPond).

> *Ce `\mergeNotes` provoque dans le moteur l'écriture de `\mergeDifferentlyHeadedOn \mergeDifferentlyDottedOn`. On pourra préférer utiliser ces deux marques, ou seulement l'une d'entre elles, dans le détail, pour un résultat différent attendu.*

Arpège vers accord tenu

VERSION SIMPLIFIÉE

Pour obtenir l'image :



… utiliser le code simplifié suivant :

```
\tieWait \stemUp aes’8~ \tieDown c~ f~ <aes, c f aes>4
```

On peut le simplifier encore en précisant par une capitale « U » ou « D » à la fin de `\tieWait` la direction des liaisons (tie) et par voie de conséquence des hampes (stem) si elles sont fixes comme ici. Cette lettre concernera la position des liaisons, la position des hampes étant à l’inverse. « D » signifie « down » (bas) et « U » signifie « up » (vers le haut).

Donc on peut encore simplifier le code ci-dessus par :

```
\tieWaitD aes’8~ c~ f~ <aes, c f aes>4
```

> *Note : Les liaisons ne semblent malheureusement pas obéir à tous les coups.*

VERSION AVEC DURÉE DE NOTES

On peut également obtenir une version avec les notes « doublées » avec leur durée, ainsi :



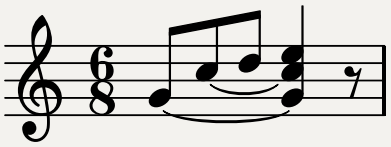
Ci-dessus, comment comment le Sol et le Do indiquent *explicitement* leur durée contrairement à l’exemple ci-dessus qui le suggérait seulement.

L'ordre de construction de cette cellule est importante, on la détaille ci-dessous pour arriver jusqu'au code voulu.

D'abord, il nous faut les croches liées à l'accord, c'est ce que nous avons vu plus haut. Nous allons utiliser `\tieWait` mais en ajoutant un « D » pour indiquer que les liaisons (« tie ») doivent être en bas (sinon, à cause de la position haute des notes, elles seraient au-dessus). On obtiendra donc :

```
--time 6/8

\tieWaitD g'8~ c~ d <g, c e>4 r8
```

On obtient dans un premier temps : 

On ajoute ***à la suite de ce code*** les notes avec leur durée explicite en considérant des voix simultanées :

```
--time 6/8

<< { \tieWaitD g'8~ c~ d <g, c e>4 r8 } \\ { g4. s } \\ { s8 c4 s4. } >>
```

Ce code produira : 

Il faut donc corriger les erreurs :

- mettre la hampe (stem) de Do dans le bon sens,
- « merger » les notes pour ne pas doubler les têtes.

On arrive donc au code :

```
--time 6/8

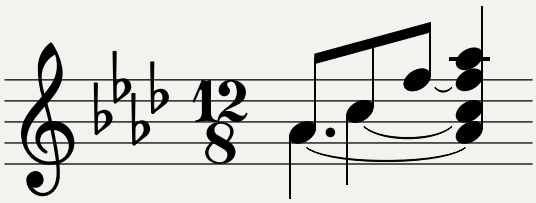
<< { \tieWaitD g'8~ c~ d <g, c e>4 r8 } \\ { \mergeNotes g4. s }
\\ { \mergeNotes \stemDown s8 c4 s4. } >>
```

… qui produit le code suivant :

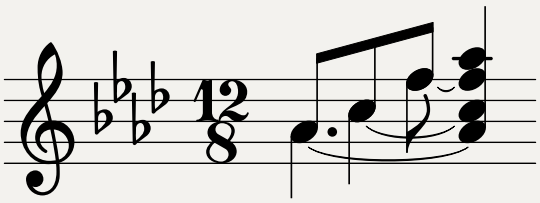


DESCRIPTION D'UN PROBLÈME

Sauf erreur de notre part, il est extrêmement difficile, avec LilyPond, d'obtenir l'image suivante, qui ressemble à la précédente, mais se différencie au niveau de la dernière note de l'arpège, le Fa en croche lié à l'accord :



Le mieux que nous puissions réussir :



> *Notez ci-dessous la note Fa qui possède sa hampe doublée.*

Pour obtenir ce code, nous devons utiliser le code très couteux suivant :

```
\mergeNotes << { \stemUp aes'8 c f } \\ { \set tieWaitForNote =
##t << { \stemDown aes,4.~ } { s8 \tieDown \stemDown c4~ } { s4
\tieDown \stemDown f8~ } { s4. \stemUp <aes, c f aes>4 } >> } >>
```

Il utilise notamment la propriété `tieWaitForNote` qui permet de lier des notes à un accord de façon simple.

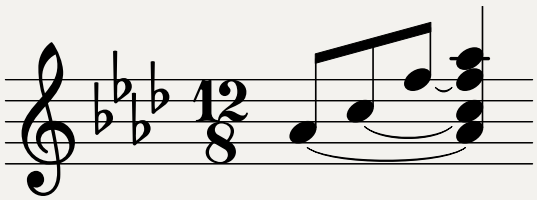
Si l'**on ne tient pas à donner explicitement la durée exacte de chaque note**, on peut simplement utiliser :

```
\set tieWaitForNote = ##t \stemUp aes'8~ \tieDown c~ f~ <aes, c f aes>4

# Et donc la version réduite :

\tieWaitD aes'8~ c~ f~ <aes, c f aes>4
```

… qui produira :



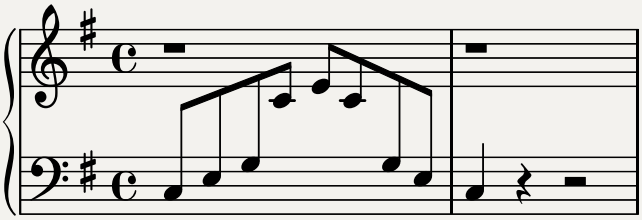
> *Voir en annexe les étapes pour obtenir ce code pour bien le comprendre.*

Changement de portée

Pour inscrire provisoirement les notes sur la portée au-dessus ou en dessous dans le mode « piano », utiliser `\up` et `\down`. Par exemple :

```
--piano
r1 r
c8 e g \up c e c \down g e c
```

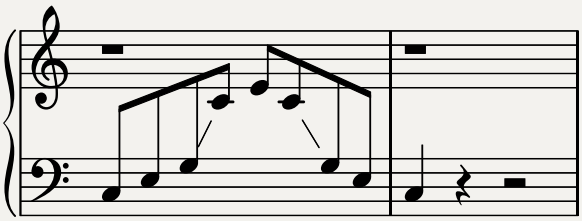
… produira :



On peut indiquer explicitement le lien entre deux notes qui changent de portée (par un trait) en ajoutant l'indication `\showStaffSwitch`

```
--piano
r1 r1
c8 e g \showStaffSwitch \up c e c \down g e c4 r r2
```

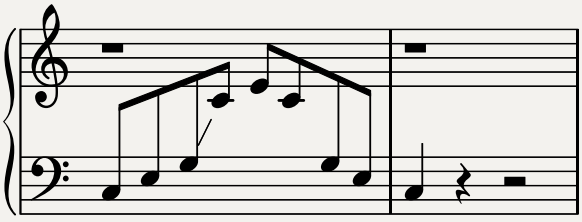
… produira :



Pour interrompre la marque de changement de portée, on utilise à l'inverse `\hideStaffSwitch`. Par exemple, le code :

```
--piano
r1 r1
c8 e g \showStaffSwitch \up c e c \hideStaffSwitch \down g e c4 r
r2
```
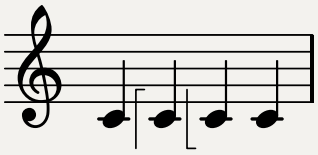
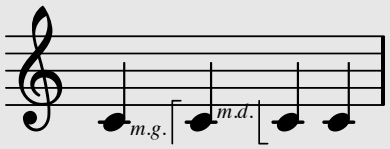
…produira :



Note jouée par autre main (piano)

Pour indiquer, par exemple dans une partition pour piano, qu'une note de la portée supérieure est jouée par la main gauche ou qu'une note de la portée inférieure est jouée par la main droite, on utilise les marques simplifiées **`\md`** (pour « Main Droite » — ou **`rh`** si la langue n'est pas française) ou **`\mg`** (pour « Main Gauche » — ou **`\lh`** en anglais). Ou leur équivalent avec le texte en ajoutant « t » à la fin de la marque.

> *Note 1 : pour le moment, le crochet se trouve un peu trop près de la note précédente lorsque l'on ne met pas le texte (donc avec `md` ou `mg`). On conseille d'utiliser toujours le texte pour la clarté, pour éloigner la note précédente.*
>
> *Note 2 :* ***on ne peut pas utiliser cette marque avec les accords pour le moment.***
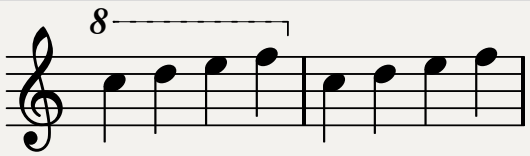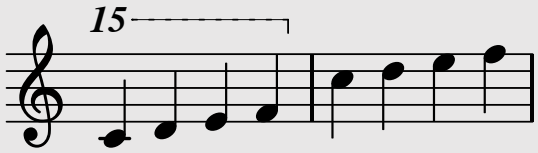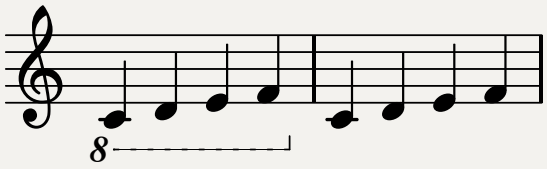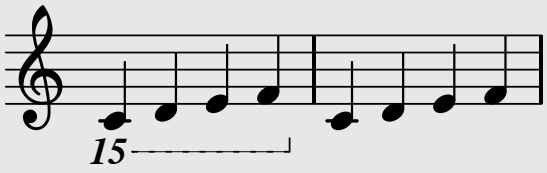
CODE	IMAGE
`c \mg c \md c c`	
`c \mgt c \mdt c c`	

Marques d'octave

Pour inscrire la marque d'octave, on peut utiliser `\8ve` (descendra les notes d'une octave et ajoutera la marque), `\15ve` (descendra les notes de deux octaves et ajoutera la double marque).

On fait l'inverse avec `\-8ve` (pour remonter les notes d'une octave) et `\-15ve` (pour remonter les notes de deux octaves).

On terminera toutes les marques précédentes avec `\0ve`.
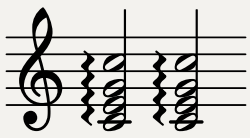
LE CODE…	… PRODUIRA :
`\8ve c'' d e f \0ve c, d e f`	
`\15ve c'' d e f \0ve c, d e f`	
`\-8ve c, d e f \0ve c' d e f`	
`\-15ve c,, d e f \0ve c'' d e f`	

Arpèges

On peut simplifier la marque `\arpeggio` par la marque **`\arp`**.

```
1  <c e g c>2\arpeggio <c e g c>2\arp
```

… produira :



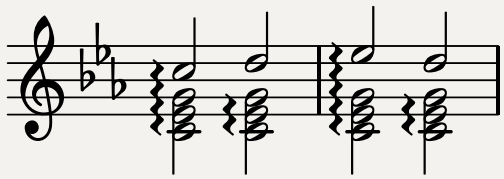
Notes connectés par \arp

Lorsque plusieurs voix doivent être connectée par la même marque d'arpège, il faut utiliser la marque **`\connectArp`** au début de la mesure ou de la section.

Par exemple :

```
--tune Cm
\connectArp
<<
  c'2\arp d ees\arp d
//
  <c, ees g>2\arp <c ees g>\arp <c ees g>\arp
  <c ees g>\arp
>>
```
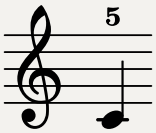
… produira :



Doigtés

Dans l'usage courant, les doigts en *mus* se marquent comment en lilypond, à l'aide de -. Par exemple, pour un cinquième doigt sur un Do :

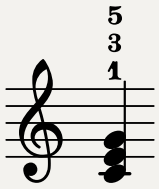
```
c-5
```



On peut indiquer plusieurs doigtés en multipliant les « `-x` ». On peut par exemple écrire un accord :

```
<c e g>-1-3-5
```
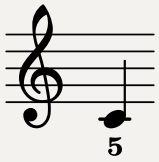
… qui produira :
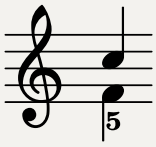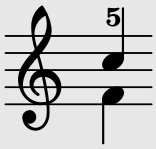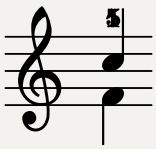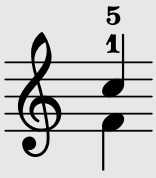


Si on doit forcer le doigté à se placer en dessous, on utilise :

```
c_5
```

… qui produira :



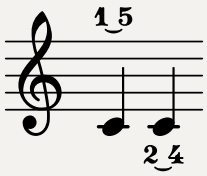
Si on doit forcer le doigté au-dessus, on utilise le `^` traditionnel. Cela se produit par exemple avec l'écriture à plusieurs voix :

LE CODE…	PRODUIRA…
`<< \stemDown f // \stemUp c'-5 >>` alors que le « 5 » devrait être au-dessus du Do.	
On utilise alors le code `<< \stemDown f // \stemUp c'^5 >>` pour forcer la marque au-dessus du Do.	
Mais si on a deux doigts avec le code : `<< \stemDown f-1 // \stemUp c'^5 >>` on obtient une collision du 5 et du 1 au-dessus de la portée.	
Il faut donc utiliser le code `<< \stemDown f // \stemUp c'^1^5 >>`	

DOIGTÉ DE SUBSTITUTION

Un raccourci permet d'écrire simplement les doigtés de substitution, à l'aide d'un **tilde** :

```
c-1~5 c_2~4
```



> *Note : On part du principe que le doigté est forcément un nombre de 1 à 5. Si pour une raison ou une autre (l'arrivée d'extraterrestres à mains de 8 doigts) le doigté devrait être différent, il faut écrire le code LilyPond explicite. Voir pour ça la page Doigtés.*

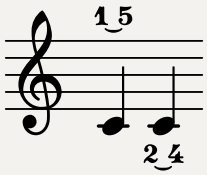
SUPPRESSION DES DOIGTÉS

Les doigtés peuvent être supprimés ponctuellement de la gravure en utilisant l'option `--no_fingers`.

Par exemple :

```
c-1~5 c_2~4
```

… produira :
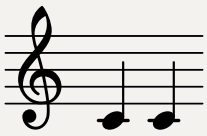


Mais :

```
--no_fingers

c-1~5 c_2~4
```

… produira :



Parenthèses

On peut ajouter des parenthèses autour d'un élément quelconque à l'aide du code générique :

`[<taille>]\([<padding ?]<code>\)`

Sans définir de taille ou de padding, le code donne simplement :
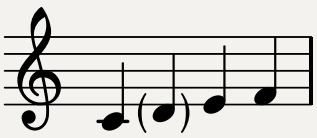
`\(<code>\)`

> *Ne oublier les balances « `\` » avant les parenthèses.*

Par exemple :

```
c \(d\) e f
```

… produira :
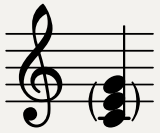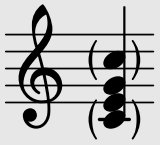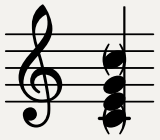


PLUSIEURS NOTES

Noter qu'il est impossible, pour le moment, de mettre plusieurs notes successives à la suite. Par exemple, le code suivant ne produira aucun résultat particulier :

```
c \(d e f\)  # :( PAS DE PARENTHÈSES
```

DIVERS CAS SPÉCIAUX

On peut mettre entre parenthèses n'importe quel élément musical et non musical.

DESCRIPTION	CODE	RÉSULTAT
Un accord complet	`\(<c e g>\)`	
Des notes dans l'accord	`<\(c\) e g \(c\)>`	
À l'intérieur d'un accord, on ne peut pas mettre des tailles ou des paddings différents. C'est toujours le premier qui l'emportera sur tous les autres.	`<-1\(-0.5 c\) e g 5\(5c\)>`	
Pour des petites notes	`\(\gr(b8)\) c`	
Petites notes avec parenthèses personnalisées (**ne fonctionne pas pour le moment**)	`5\ (-0.5\gr(b8)\) c`	

CAS PARTICULIER DES PETITES NOTES

Pour les *petites notes* (les *grace notes*), penser à bien mettre les parenthèses autour de l'ornement, pas à l'intérieur :

```
\gr(\(b8\))   # :( ne fonctionnera pas

\( \gr(b8) \) # :) OK !
```

Fonctions ruby (mode expert)

En mode expert, on peut produire du code de toute pièce avec des fonctions ruby. On peut penser par exemple au premier prélude de Bach dans le premier clavier tempéré. Presque d'un bout à l'autre on retrouve le même motif qu'il serait laborieux de répéter. Il se présente dans la première mesure de cette manière, écrit de toutes notes :

```
-> score
r8 g'16 c e g, c e r8 g'16 c e g, c e
<< r16 e8.~ e4 // c2 >> << r16 e8.~ e4 // c2 >>
```

Voilà la méthode qu'il est possible de créer dans un fichier qu'on appellera par exemple **`module_image.rb`** (peu importe son nom, c'est son extension qui fait qu'il sera chargé par ScoreImage.

```
module ScoreImage # ce nom est impératif

  def motif(basse, contrebasse, notes_sup)
    ns = notes_sup.split(' ')
    <<~TEXT
    % #{ns[0]}'16 #{ns[1]} #{ns[2]} #{ns[0]}, #{ns[1]} #{ns[2]}
%2
    % << r16 #{contrebasse}8.~ #{contrebasse}4 // #{basse}2 >>
%2
    TEXT
  end

end
```

Il nous suffit maintenant d'appeler la méthode `#motif` dans le code `.mus` à l'aide du préfix **`fn_`** (« fn » comme « fonction »).

```
-> score
fn_motif("c", "e", "g c e")
```

> *Noter que contrairement à du pur ruby, il faut obligatoirement utiliser les parenthèses pour délimiter les arguments **même lorsqu'il n'y en a pas**.*

Constantes notes

On peut utiliser les constantes au lieu de guillemets pour simplifier l'écriture :

```
-> score
fn_motif(c, e, g)
```

Parenthèses dans les arguments

Si les arguments contiennent des parenthèses, pour éviter toute confusion, on utilise des double-parenthèses pour délimiter les arguments de la fonction :

```
fn_motif(( "a( b c d)" ))
```

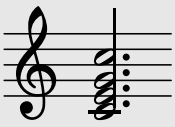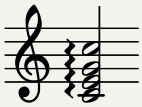
Langage LilyPond (aide mémoire)

Ci-dessous la syntaxe propre à Lilypond, pour mémoire.
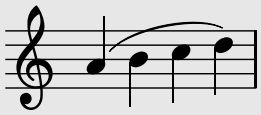
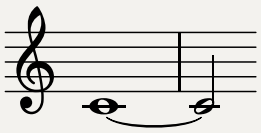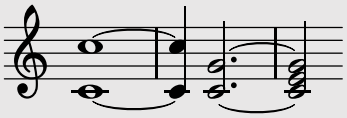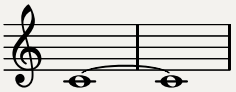
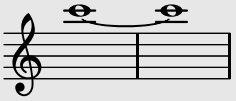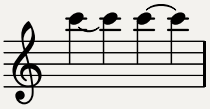
Altérations

OBJET	CODE	DESCRIPTION
`#`	**`is`**	Par exemple `fis` pour fa dièse
`b`	**`es`**	Par exemple `ees` pour mi bémol
`x`	**`isis`**	Par exemple `gisis` pour sol double-dièses
`bb`	**`eses`**	Par exemple `aeses` pour la double bémols
	`!`	Altération de prudence. Par exemple `a!` pour forcer la marque du bécarre lorsque le La a été diésé dans la mesure précédente.
	`?`	Altération entre parenthèse. Par exemple `a?` pour mettre un bécarre avant le La entre parenthèses pour bien indiquer qu'il est bécarre.
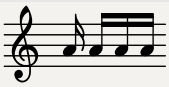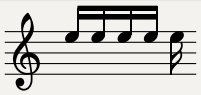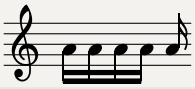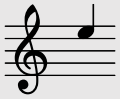
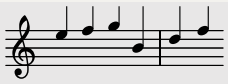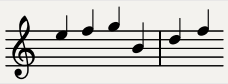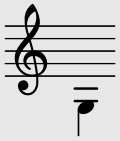
Accords

OBJET	CODE	DESCRIPTION
	`<notes >duree`	Bien noter que la durée est à l'extérieur de l'accord. Noter aussi que c'est la hauteur de la première note qui détermine la hauteur de référence pour la note suivante
Exemple	**2.**	
Arpège	`<c e g>\arp`	Il suffit d'ajouter la marque `\arp` après l'accord (et la durée) pour obtenir un arpège. 

Liaisons

OBJET	CODE	DESCRIPTION
Liaisons de jeu	`note1( autres notes)`	
exemple	`a'( b c d)`	
forcer en dessous	`a'_( b c d)`	
forcer au-dessus	`a'^( b c d)`	
Forcer en haut	`\slurUp`	Pour revenir au comportement par défaut : `\slurNeutral`
Forcer en bas	`\slurDown`	Pour revenir au comportement par défaut : `\slurNeutral`
Placement par défaut	`\slurOff`	`\slurNeutral` est la formule d'origine de LilyPond
Liaison de durée	`note~ note`	
Exemple simple	`c1~ c2`	
Exemple avec des accords	`<c c'>1~ <c c'>4 <c~ g'~>2. <c e g>2`	
Forcer la liaison en haut	`\tieUp`	
Forcer la liaison en bas	`\tieDown`	
Revenir au comportement par défaut	`\tieOff`	 Dans l'exemple ci-dessus, le `\tieOff` est inséré entre les deux notes. `\tieNeutral` est la formule d'origine de LilyPond

Attache des hampes des notes

OBJET	CODE	DESCRIPTION
Forcer l'attache	`note[ notes]`	
Exemple	`a'16[ a a a a a a a]`	
Empêcher l'attache d'une note	a'16\noBeam a a a	
Forcer l'attache vers le haut	`note^[ notes]`	
Exemple	`e'16^[ e e e] e`	
Forcer l'attache vers le bas	`note_[ notes]`	
Exemple	`a'16_[ a a a] a`	
Forcer une hampe seule en haut	`\stemUp e'4 \stemOff e'4^[]`	
Forcer les hampes de plusieurs notes non attachées (noires et blanches)	`\stemUp e'4 f g f \stemOff e'4^[] f^[] g^[] f^[]`	N1 : Noter que si plusieurs notes (plusieurs noires par exemple) doivent être traitées ensemble et que ce ne sont pas les mêmes hauteurs, il ne faut pas utiliser `e'4^[ f g f]` car dans ce cas tous les hauts de hampes s'aligneraient à la même hauteur. Il est impératif d'utiliser le code ci-contre. Cf. ci-dessous.
	`e'4^[ f g b, d f]`	
	`e'4^[] f^[] g^[] b,^[] d^[] f^[]`	
Forcer une hampe seule en bas	`g4_[]`	
		Pour plusieurs noires ou plusieurs blanches, cf. la note N1 ci-dessus.
Forcer le regroupement des notes		cf. Regroupement des hampes

Voir la page suivante pour la gestion des deux en même temps :
https://lilypond.org/doc/v2.19/Documentation/notation/beams.fr.html

Il semble qu'il faille utiliser :

`\override Beam.auto-knee-gap = #<INTEGER>`

Anacrouse
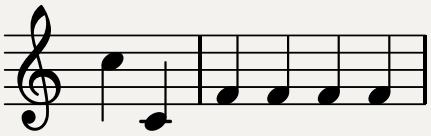
Démarrage en levée de la mélodie, sans utiliser de silences invisibles avant (`r`) :

`\partial <durée de l'anacrouse>`
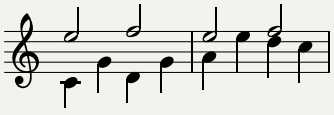
Par exemple, pour avoir deux temps avant la première mesure :

```
\partial 2 c'4 c, f f f f
```

Produira :



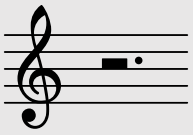
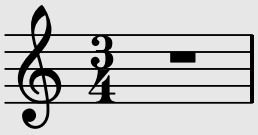
Voix simultanées

OBJET	CODE	DESCRIPTION
	`<< { note note note } \\ { note note note } >>`	Le plus clair et le plus simple est d'utiliser des variables à la place des notes. La hauteur de la première note du second membre est calculée à partir de la première note du premier membre
Version simplifiée	`<< note note // note note >>`	C'est la version simplifiée de la précédente.
Exemple	`<< { e'2 f e f } \\ { c,4 g' d g a e' d c } >>`	
Snippet	`2v`	

Dans cette formule, les deux voix auront leur propre 'voice'.
Mais il existe d'autres possibilités (cf. le mode d'emploi)

Silence sur mesure entière

Avec LilyPond, on met un silence sur une mesure entière avec un « R » (« r majuscule »).

`r2.`	`--time 3/4` `R2.`
	
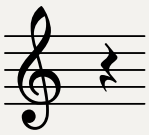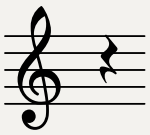
Placement des silences

Avec LilyPond, on peut placer les silences de façon très précise avec la formule :

```
1  <note><octave><duree>\rest
```

Le silence sera alors placé à l'endroit de la note (la « tête » du silence sera sur la tête de la note).
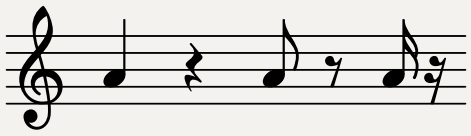
Par exemple, pour placer un silence sur le Do 4 (ou Do 5 en anglais)

PLACEMENT NATUREL	PLACEMENT PRÉCIS
	
`r4`	`c'4\rest`

La durée courante influe sur la durée du silence, comme on peut le voir avec le code :

```
1  a'4 b\rest a8 b\rest a16 b\rest
```

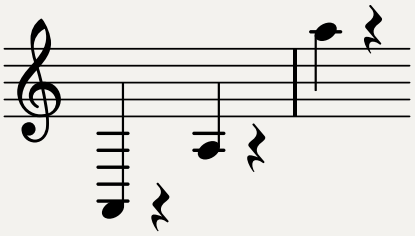
… qui produira :



De la même manière, l'octave courante influe sur la hauteur du silence :

```
a,4 a\rest a' a\rest a'' a\rest
```

… produira :
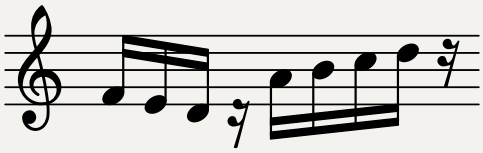


Il faut noter un point important : en matière d'octave, la note du silence est influencée par l'octave précédente et influe sur l'octave (les notes ou les silences) suivants.

Ainsi, le code :

```
f16 e d d\rest a' b c d d\rest
```

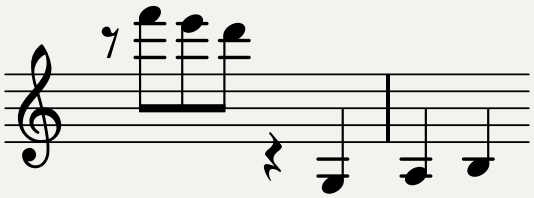
… produira :



… qui montre l'influence de l'octave et de la durée précédente sur le silence, tandis que le code :

```
c''8\rest f e d c,,4 g a b
```

… produira :



… qui montre l’influence de la durée et de la hauteur sur les notes suivants le silence.

Variable (aka « Definitions »)

On peut créer des « définitions » qui pourront être ensuite utilisées dans l'expression LilyPond fournie. Ceci permet d'écrire de façon plus modulaire et de pouvoir composer des segments différents très facilement.

Typiquement, on peut faire une définition pour chaque mesure. Dans une partition pour piano du premier mouvement de la Sonate facile de Mozart, on pourrait avoir par exemple :

```
--piano
--barres
--times

# La définition de la première mesure
# <nom de la variable-définition>
# <notes de main droite>
# <notes de main gauche>
mes1=
c'2 e4 g
c8 g' e g c, g' e g

mes2=
b'4.( c16 d) c4 r
d8 g f g d g f g

# Un segment comprenant ces deux mesures se définirait par :
-> mesures-1-a-2
mes1 mes2
mes1 mes2
```

Déclaration de la variable-définition

Sur une seule ligne, un nom ne contenant que des lettres majuscules ou minuscules et des chiffres, terminé par un ou deux signes « égal ».

- Avec **un seul signe égal**, c'est une **variable locale** (elle sera supprimée tout de suite après la réalisation de la première image).
- Avec **deux signes « égal »**, c'est une **variable globale** qui restera utilisable jusqu'à la fin du fichier.

La définition, qui peut tenir sur plusieurs lignes (une — monodie — ou deux — piano — pour le moment) et contenir des options, se termine à la première ligne vide rencontrée.

Par exemple :

```
1
2 mesure1=
3 c d e f g a b c
4 c b a g f e d c
5
6 # La ligne vide ci-dessus met fin à la définition
```

Variable dynamique

Depuis 2024, les variables sont « dynamique », c'est-à-dire qu'elles peuvent varier en fonction de paramètres. À commencer par leur hauteur.

Hauteur de la variable

Par exemple, soit la variable `maVar` définie par :

```
maVar=
c d e f g
```

Alors si on utilise dans le code :

```
-> partition
maVar' maVar maVar,
```

Cela produira les notes :

```
\relative c' { c' d e f g } \relative c' { c d e f g } \relative c' { c, d e f g }
```

C'est-à-dire que les marques « `'` » et « `,` » , comme pour les notes, permettent de définir la hauteur où sera jouer la variable.

Répétition de la variable

De la même manière, on peut définir le nombre de fois où la variable doit être répétée avec `*N`. Par exemple :

```
-> partition
maVar*4
```

Produira :

```
\relative c' { c d e f g c d e f g c d e f g c d e f g }
```

Utilisation d'un rang de variables

L'utilisation des variables-définitions prend tout son intérêt avec la **définition de l'expression par rang de variables**.

Très simplement, cela signifie que si on déclare ces variables-définitions :

```
mes1==
... notes ...

mes2==
... notes ...

mes3==
... notes ...

mes4==
... notes ...
```

… on peut déclarer facilement un segment (une image, donc) avec :

```
mes1<->4
```

Cela signifie que le segment sera constitué des mesures 1 à 4.

Définition par section

Comprendre avant tout qu'il y a deux façons d'utiliser les variables. On peut utiliser une variable pour définir les mesures d'un instrument particulier, monodique. On assemblera ensuite les variables pour former l'ensemble. Par exemple :

```
flute=
c d e f g ...

basse=
c c c c ...

-> ensemble
flute
basse
```

On peut, à l'inverse, utiliser les variables pour définir des sections, en mettant toutes les voix dedans. Par exemple :

```
intro=
c d e f g ...
c c c c c ...

couplet=
e g e g e ...
e e e e e ...

refrain=
f a c f ....
f f f f ....

-> morceau
intro couplet couplet refrain couplet
intro couplet couplet refrain couplet
```

> *Noter, ci-dessus, qu'en l'absence de la définition du nombre de voix, on doit répéter la ligne.*

On peut bien entendu utiliser les deux types de définition en même temps :

```
intro_flute=
c d e f g ...

```

```
couplet_flute=
e g e g e ...

refrain_flute=
f a c f ...

intro_basse=
c c c c ...

couplet_basse=
e e e e ...

refrain_basse=
f f f f ...

intro=
intro_flute
intro_basse

couplet=
couplet_flute
couplet_basse

refrain=
refrain_flute
refrain_basse

-> morceau_per_section
intro couplet refrain

# Et/ou

flute=
intro_flute couplet_flute refrain_flute couplet_flute ...

basse=
intro_basse couplet_basse couplet_basse refrain_basse ...

-> morceau_per_instrument
```

```
flute
basse
```

> *Noter qu'on ne peut pas utiliser en même temps, en mode --piano, des définitions d'une seule main et des définitions des deux mains. Si on adopte un mode, il doit être utilisé pour tout l'extrait.*
> *Mais deux scores différents peuvent utiliser deux modes qui diffèrent, par exemple :*

```
# Un extrait avec définition d'une seule main

--piano

mg1=
c1 e g

md1=
g8( a b c) c2

-> essai_par_mains
mg1
md1

mgd1=
c1 e g
g8( a b c) c2

-> essai_deux_mains
mgd1

```

NOTA BENE

Pour utiliser la fonctionnalité précédente avec autre chose que le piano, il faut définir explicitement le nombre de voix.

L’exemple ci-dessous ne fonctionnera pas ou, plus exactement, ne produira que la voix supérieure.

```
md1=
c2 e4 g

mg1=
g8( a b c) c2

md2=
b4. c8 c4 r

mg2=
d g f g c, g' e g

# Définitions à voix multiples
tout=
md1 md2
mg1 mg2

-> fichier
tout <====== NE FONCTIONNE PAS (PAS DE DÉFINITION DU NOMBRE DE VOIX)

```

Le code ci-dessus fonctionnera si on définit à l’aide de l’option `staves_keys` le nombre de voix :

```
--staves_keys G,G   <===== DÉFINITION DU NOMBRE DE VOIX

md1=
c2 e4 g

mg1=
g8( a b c) c2

md2=
b4. c8 c4 r
```

```
11
12 mg2=
13 d g f g c, g' e g
14
15 # Définitions à voix multiples
16 tout=
17 md1 md2
18 mg1 mg2
19
20 -> fichier
21 tout <====== FONCTIONNE !!!
```

Annexe

Fichier de coloration syntaxique pour Sublime Text

Ce code est à placer dans `/Library/Applications Support/Sublime Text 3/Packages/User/music-score.sublime-syntax`

> *Note : depuis le crash de 2021, ce fichier fait partie des backups universels.*
>
> *Mais il ne fonctionne plus bien non plus*

```
1 %YAML 1.2
2 ---
3 # See http://www.sublimetext.com/docs/3/syntax.html
4 file_extensions:
5   - mus
6 scope: source.music
7
8 contexts:
```

```
  # The prototype context is prepended to all contexts but those setting
  # meta_include_prototype: false.
  prototype:
    - include: comments

  main:
    # The main context is the initial starting point of our syntax.
    # Include other contexts from here (or specify them directly).
    - include: numbers
    - include: notes

  numbers:
    - match: '[^,a-g][0-9]+(\-[0-9]+)?\b'
      scope: constant.numeric.example-c

  notes:
    # Les notes
    - match: "\\b[a-gr](es|is)?(es|is)?[',](16|32|64|128|1|2|4|8)?"
      scope: music.note
    - match: "\\b[a-gr](es|is)?(es|is)?\\b?[',]?(16|32|64|128|1|2|4|8)?\\b"
      scope: music.note
    # Les accords
    - match: '<.+?>(16|32|64|128|1|2|4|8)?'
      scope: music.note.chord
    # Les liaisons
    - match: '\( .+?\)'
      scope: music.note
    # Les options qu'on peut trouver
    - match: '\-\-(verbose|keep|barres|time|piano|only_new|stop|start|open|big-|mini-|biggest)(hspace)?( (OFF|ON))?\b'
      scope: music.option
    # Numéro de mesure et proximité
    - match: '\-\-(mesure|proximity) [0-9]+\b'
```

```
      scope: music.mesure.numero
    # Tonalité
    - match: '\-\-(tune|key) [a-zA-Z][#b]?'
    # Format de page
    - match: '\-\-page [a-zA-Z0-8]+'
      scope: music.mesure.numero
    # Définition (variable)
    - match: '[a-zA-Z0-9_]+\=\=?'
      scope: music.mesure.numero
    # Nom de fichier
    - match: '^\-> (.+)$'
      scope: music.output.name
    # Les clés
    - match: '\\clef? ("(treble|bass|ut)"|G1|G|F3|F|UT1|UT2|UT3|UT4|UT5)'
      scope: music.clef
    # Les barres de mesure
    - match: ':?\|:?'
      scope: music.mesure.barre
    # Special words
    - match: '\\(trill|slashedGrace|grace|appoggiatura|acciaccatura|break) ?'
      scope: music.clef

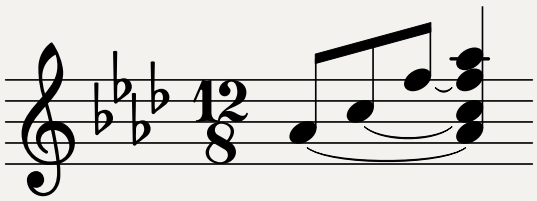
  inside_string:
    - meta_include_prototype: false
    - meta_scope: string.quoted.double.example-c
    - match: '\.'
      scope: constant.character.escape.example-c
    - match: '"'
      scope: punctuation.definition.string.end.example-c
      pop: true

  comments:
    # Comments begin with a '# ' and finish at the end of the line.
    - match: '# '
      scope: punctuation.definition.comment.example-c
      push:
```

```
77          # This is an anonymous context push for brevity.
78          - meta_scope: comment.line.double-slash.example-c
79          - match: $\n?
80            pop: true
81
```

Étapes pour « arpège vers accord tenu »

L'image suivante :



… est obtenue à partir du code :
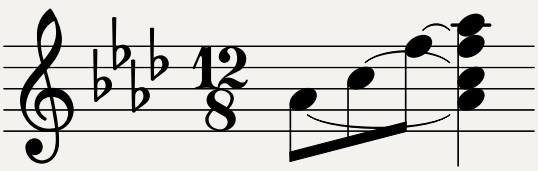
```
1  \set tieWaitForNote = ##t \stemUp aes'8~ \tieDown c~ f~ <aes, c f
   aes>4
```

> *Voir la version simplifiée.*

Pour obtenir ce code, nous avons procédé ainsi. Nous sommes partis de :

```
1  \set tieWaitForNote = ##t aes'8~ c~ f~ <aes, c f aes>4
```

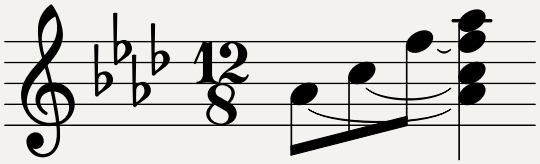
… qui a produit :



Pour avoir les liaisons dans le bon sens avec `\tieDown`, nous avons fait :

```
1  \set tieWaitForNote = ##t aes'8~ \tieDown c~ f~ <aes, c f aes>4
```

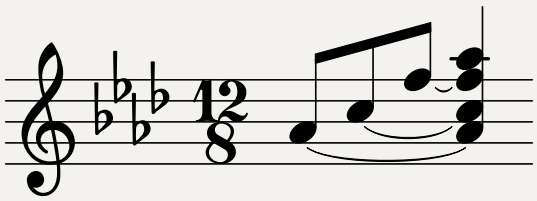
… qui a produit :



Et enfin nous avons mis les hampes dans le bon sens avec `\stemUp` :

```
\set tieWaitForNote = ##t \stemUp aes'8~ \tieDown c~ f~ <aes, c f aes>4
```

… qui a produit :



Score Builder

L'application ***Score-Builder*** permet de produire très facilement du code Mus, en voyant le résultat immédiatement. En plus, elle permet aussi d'avoir une partition de référence contenant la partition à produire.

Pour utiliser *ScoreBuilder*, si l'application est installée, il suffit d'ouvrir un Terminal à un dossier (contenant par exemple la partition originale) et de jouer `score-builder` (ou plus simplement `score-b` suivi de la touche tabulation). L'assistant prendra les choses en main pour vous faire commencer.

Tests

Score-Image est testée par des comparaisons entre les images attendues et les images produites, pour être sûr d'un résultat optimum. Cf. le dossier `tests/checksums_tests` qui contient tous les fichiers.

Pour lancer tous les tests :

```
score-image tests _
```

Pour filtrer les tests à jouer :

```
score-image tests /<filtre régulier>/
```

Pour ne lancer que les tests d'un dossier :

```
score-image tests _ -dir=dossier
```

Et en les filtrant :

```
score-image /ceuxla/ -dir=mon/dossier
```

En plus de l'image produite, les tests s'assurent aussi que toutes les statistiques soient correctes, ce qui permet un tour d'horizon parfait.

Pour de plus amples informations, lire le fichier `tests/_ReadMe_.md`

Collisions verticales

Cf. la page dans le manuel LilyPond.